ORIGINAL ARTICLE

EvoBagNet: a decomposition-driven bagging ensemble with evolutionary hyperparameter optimization for robust stock price prediction

Umar Bashir · Kuljeet Singh · Vibhakar Mansotra · Akib Mohi Ud Din Khanday · Mehdi Neshat

Received: 11 February 2025 / Accepted: 7 October 2025 / Published online: 3 February 2026

Abstract

The rapid economic growth of recent years has led to a surge in stock market participation, necessitating the need for accurate stock price predictions to mitigate investment risks and maximize returns. However, the dynamic nature of stock prices and their intrinsic volatility pose significant challenges to traditional statistical and machine learning (ML) models, which often struggle with overfitting, poor robustness, and limited generalization. To address these challenges, this study introduces a novel framework: EvoBagNet, an evolutionary Bagging ensemble learning model specifically designed for robust and high-accuracy stock price prediction. EvoBagNet is a scalable and efficient ensemble framework combining an Extra tree-based model, categorical boosting (CatBoost), and Light Gradient Boosting Machine (LGBM) as part of a bagging ensemble technique to enhance predictive performance. The framework incorporates Complete Empirical Mode Decomposition (CEEMD) to decompose time series data into intrinsic mode functions (IMFs) across varying frequency spectra, allowing for a more granular analysis of temporal patterns. Hyper-parameter tuning is conducted using a fast, single-objective evolutionary algorithm designed to converge efficiently on optimal configurations for the ensemble model. The framework is evaluated on datasets from nine prominent IT sector companies, employing six rigorous evaluation metrics to comprehensively assess performance. Experimental results highlight EvoBagNet's superior accuracy, robustness, and scalability, outperforming state-of-the-art models across diverse scenarios and datasets. EvoBagNet demonstrated exceptional prediction accuracy across all datasets, achieving performance scores of 97.0% ± 0.7, 98.3% ± 0.5, 97.3% ± 0.8, 97.4% ± 0.6, 97.0% ± 1.0, 98.6% ± 0.4, 98.8% ± 0.4, 91.7% ± 1.2, and 98.4% ± 0.3 for Tech Mahindra, Mindtree, Infosys, Wipro, TCS, Mphasis, L&T Tech, HCL, and Coforge, respectively. These results highlight EvoBagNet's potential as a powerful tool for stock price forecasting, offering significant implications for informed investment strategies and financial decision-making.

Keywords Bagging ensemble learning · Hyper-parameter tuning · Stock prediction · Evolutionary algorithm

Umar Bashir, Kuljeet Singh, Vibhakar Mansotra, Akib Mohi Ud Din Khanday, Mehdi Neshat have contributed equally to this work.

1 Introduction

A stock market, often known as the equity market, comprises several global stock exchanges. It is a snapshot of the future growth of the economy as well as the company's expectations. Buying and selling the shares at the right time increases the capital and generates a good return for investors [1]. As per the Efficient Market Hypothesis (EMH) [2], stock market prices imitates total present information. It facilitates the price discovery of corporate shares and serves as a barometer of the overall economy. Since there is a large number of stock market participants, one can usually expect fair pricing and a high level of liquidity as market participants compete for the best price with transparency in transactions. Over the past few years, interest in stock trading has increased. The stock market has also undergone significant changes due to technological advancements and the rise of online trading. It is considered a dynamic and chaotic financial system [3] because its behaviour is influenced by unpredictable factors, including investor expectations, and political situations within the country. Stock market data is a non-linear and variable time series, i.e., a collection of data calculated over time to track various activity statuses [4]. This complex financial system encompasses a diverse range of companies and their respective stocks, along with price adjustments that fluctuate over time for each company. Stock market prediction is the process of estimating the future value of a company's commodity or any other financial instrument being traded on the stock exchange. According to [5], financial time series data prediction is regarded as a notoriously challenging task for statistics experts and finance. Every investor plans to increase the profit from their investments and decrease the correlated risks [6]. A successful prediction can result in substantial gains for both the seller and the buyer and can be carried out through in-depth analysis of past data. According to [7], there are two approaches to stock forecasting. One is fundamental analysis, which depends on business methodology and basic data such as yearly growth rates, expenses, and market position. The second approach is technical analysis, which focuses on historical stock values.

Today, the world needs an automated and accurate prediction system. Predicting the stock market in the financial system is challenging but rewarding. Typically, in the stock market, a vast amount of structured and unstructured heterogenous market data is generated. Due to the exponential growth of data in the financial market, it is very complex to analyze using traditional statistical models. Analysts and researchers have attempted to solve this problem by applying various statistical approaches, but these methods may not yield more accurate results. Ultimately, they determined that data mining was the most effective solution for handling a large volume of structured and unstructured, heterogeneous data. With the advancement of Machine Learning approaches, there has been significant progress in automating the stock market prediction process [8]. However, there is still scope for the system to produce more accurate results. With the development of technologies like ML, the ability to predict the stock market has improved significantly. In machine learning, In machine learning, models are trained on historical stock data to predict future stock prices or movements.

1.1 Major technical contributions

The primary contribution of this study is the development of a novel hybrid forecasting model, EvoBagNet, synthesizing ensemble bagging learning, signal decomposition, and evolutionary computation for enhancing stock price predictive performance under turbulent financial environments. The integrated structure is designed to overcome the shortcomings of individual machine learning models, particularly in handling noisy, nonlinear, and high-frequency stock data.

Stock price time series are inherently non-stationary and typically consist of multi-frequency components based on both high-frequency noise and the long-run market trend. CEEMD counters this by decomposing the original time series into Intrinsic Mode Functions (IMFs). IMFs demix oscillatory modes at different frequencies so that a more localized and frequency-aware learning process. Each of the decomposed fragments is subsequently addressed individually by the ensemble system, allowing EvoBagNet to more clearly observe both local anomalies and global trends than traditional models. Furthermore, EvoBagNet employs a bagging (bootstrap

aggregating) technique as its core ensemble method, utilizing various base learners, including CatBoost, LGBM, and Extra Trees. Bagging encourages stability by reducing model variance through parallel learning over bootstrapped sets. In contrast to simple ensemble methods, EvoBagNet is the first to combine multiple decomposed learners through a specially designed averaging mechanism tailored for decomposed frequency features, resulting in a multiscale combination of learned patterns. The architecture allows EvoBagNet to generalize better across multiple stock indices and temporal dynamics.

In addition, EvoBagNet makes use of a (1 + 1) Evolutionary Algorithm (EA) for hyperparameter optimization in order to enhance the predictive accuracy and robustness of the ensemble. Traditional methods, such as grid search or random search, are computationally costly and suboptimal when dealing with high-dimensional parameter space. The 1 + 1 EA employed here performs a mutation-based search around a single candidate solution based on prediction accuracy as the fitness function. The algorithm repeatedly tunes the hyperparameters of the ensemble–number of trees, learning rates, maximum depths, and subsample sizes–to converge into configurations delivering maximum out-of-sample performance. This adaptive change also improves accuracy, reduces overfitting, and shortens training time.

Finally, by incorporating CEEMD for signal decomposition, bagging for diversity of models and variance reduction, and evolutionary computation for dynamic optimization, EvoBagNet is a systematic and scalable hybrid learning pipeline. It is validated on data from nine top IT companies that are listed in the National Stock Exchange (NSE) and compared against six evaluation measures (MAPE, R^2, RMSE, MAE, etc.). The proposed framework consistently outperforms individual ML models in terms of predictive accuracy, robustness, and generalizability.

The main contributions of this work briefly are as follows:

- This study applies nine popular machine learning algorithms, including SVR, RF, Lasso Regression, NN, XGBoost, GBR, DT, LGBM, and CatBoost, to predict the future closing prices of leading IT sector stocks. Each model is rigorously evaluated using six performance metrics, providing a detailed comparison of their predictive capabilities.
- Six effective bagging ensemble models are proposed, which leverage the advantages of bagging, such as reduced variance, improved stability, and enhanced robustness against overfitting, to deliver superior predictive performance.
- The framework incorporates Complete Empirical Mode Decomposition (CEEMD) to decompose stock price time series data into intrinsic mode functions (IMFs) across varying frequency spectra.
- Additionally, the performance of EvoBagNet improved using an automatic hyper-parameter tuner based on a fast and efficient evolutionary algorithm (1 + 1EA) and enhancing their accuracy and reliability in forecasting stock closing prices for IT sector companies.

The remaining paper structure consists of a literature review (Sect. 2) summarizing previous research in the equity market. The next section is materials and methods (Sect. 3), in which different modern ML approaches and proposed ensemble bagging model (EvoBagNet) have been applied in the current study. The next section is modelling stock market analysis (Sect. 4), in which the dataset description and all the data preprocessing steps are discussed. In Sect. 5, the experimental evaluation results achieved using the proposed model EvoBagNet and nine machine-learning approaches and their comparative analysis are also outlined. Finally, in the Sect. 7, the conclusion and future scope of this study are provided.

2 Literature review

In recent years, the application of Machine Learning (ML), Deep Learning (DL), and advanced AI techniques has gained substantial attention in the domain of stock price prediction [9]. These methods offer powerful capabilities for modeling the complex, non-linear, and volatile nature of financial time series data. Traditional ML

algorithms such as Support Vector Regression (SVR), Random Forests (RF), and Gradient Boosting Machines (GBM) have been widely utilized to identify short-term market patterns [10]. Meanwhile, deep learning models–including Long Short-Term Memory (LSTM) networks, Convolutional Neural Networks (CNNs), and hybrid frameworks–demonstrate superior performance in capturing long-range dependencies and intricate temporal behaviors. Furthermore, cutting-edge AI techniques [11] such as attention mechanisms, transformers, and evolutionary optimization have been explored to enhance prediction accuracy, stability, and adaptability under varying market conditions. The following subsections present a structured review of the key contributions in this evolving research landscape.

2.1 Traditional statistical and machine learning approaches

Stock price prediction has evolved from traditional statistical approaches to leveraging advanced machine and deep learning models [12]. Modern techniques such as convolutional neural networks (CNNs) [13] and recurrent neural networks [14, 15] (RNNs) enable the extraction of spatial and temporal patterns in stock data. Generative adversarial networks (GANs) generate synthetic data to enhance training, while attention mechanisms and transformer [4] models focus on identifying key features and trends, thereby, capturing complex market dynamics. These advanced methods provide a robust framework for accurate and efficient stock price forecasting in volatile financial markets.

Various ML techniques have been employed to predict stock market movements. Dinesh and Greish [16] employed a linear regression (LR) on TCS data to forecast the closing price. Compared to other machine learning approaches, this approach offers simplicity because the predictive power of LR is limited, especially when handling intricate market dynamics. Furthermore, the study's dependence on a single dataset would limit the broad applicability of the results. This leads to some degree of shortsightedness throughout the evaluation process. To address these drawbacks, Kumar et al. [17] suggested a refined version of SVR designed especially for time series data. The grid search method has been employed to select the optimal kernel function and optimize its parameters. The model showed improved performance on eight different datasets. This approach offers a more reliable and adaptable method for anticipating the stock market. In another study [18] development and assessment of five ML techniques for precise price prediction of twelve prominent Indian companies, namely Adani Ports, Asian Paints, Axis Bank, HDFC bank, TCS, NTPC, Titan, ICICI, Maruti, Tata Steel, Kotak Bank, and Hindustan Unilever Limited has been done. Five ML techniques, including K-NN, LR, SVR, DT, and LSTM. The study found that the DL algorithm, in particular LSTM, performed better than conventional ML approaches in forecasting stock market time series data.

Investors can enhance their decision-making and portfolio management by utilizing index analysis to comprehend market dynamics, economic conditions, and emerging investment opportunities. Singh in [19] examined the performance of various ML techniques on the NSE Nifty-50 index and discovered that although ANN and LR produced comparable results, ANN required a somewhat longer training time. SVM, on the other hand, demonstrated good performance but was sensitive to the dataset size. The effectiveness of MLP, RF, and LR on two indices, the Dow Jones Industrial Average index and the New York Times index, was further examined in [20]. Although MLP performed better than the other methods within a certain range, its overall generalizability could be constrained. Torres et al. [21] employed WEKA's RF and MLP to forecast the closing price of Apple Inc. Stock. The model was trained on the previous 250 trading sessions and predicts the closing price of 251st session. Researchers aim to improve the precision and reliability of stock prices by leveraging these strategies.

2.2 Deep learning and hybrid techniques in stock prediction

Recent developments in AI, particularly in neural networks, have revolutionized several fields, including financial markets. Artificial Neural Networks (ANN) have emerged as a powerful tool for regression problems, and their performance depends on the optimization of weights and biases. Neural networks are a promising approach

to enhancing investment decision-making by analyzing the complex patterns present in stock market data. The primary focus of [22] is to forecast stock price movements using neural networks and a Backpropagation (BP) algorithm. The model predicted the closing price for the following day using a 5-day window of historical data as input. To train the BP neural network, the dataset of stock transactions was used to optimize the parameters. With a success rate of 73.29%, the study indicated that the BP neural network performed better in terms of prediction accuracy than a deep-learning fuzzy algorithm. The model showed excellent accuracy, especially when making predictions within a 15-day range. Vijh et al. [23] applied two approaches, namely ANN and Random Forest, on five different sector companies for the prediction of next-day closing price. The basic features were used to create new variables and served as input to the model. RMSE, MBE, and MAPE were used for model evaluation. It is concluded that ANN demonstrated superior performance as compared to the Random Forest. The predictive generalization of ANN has been further enhanced in [24] by the implementation of Barnacles Mating Optimizer (BMO). By incorporating more relevant features, such as high-low, close-open, MA7, MA14, MA21, and Std7, the BMO-ANN model produced promising results in terms of MSE and RMSPE. In another study [25], big data analytics approaches were employed to forecast the daily return direction of SPDR S&P 500 ETF (SPY). In another study, deep neural networks (DNNs) and conventional ANNs were applied to both untransformed and PCA-transformed data to obtain higher classification accuracy and better trading strategy performance. As the number of hidden layers increased progressively, a pattern for classifying the DNNs was identified and monitored by managing overfitting. The applications of DL techniques have been explored to identify complex patterns in the stock market data. In order to extract and learn from multiple scales, Hao and Gao [26] suggested a hybrid neural network model that combines CNN and LSTM layers. The model applied two-layer CNN and raw daily price series to extract short-term, medium-term, and long-term features. LSTM networks were applied to capture temporal dependencies in these features, and ultimately, joint representations for prediction were learned through fully connected layers. However, stock market volatility is influenced by various factors, including news, social media, and comprehensive strategies that take these external influences into account. Khan et al. in [27] examines the influence of social media and financial news on stock market prediction and to further increase the performance of stock prediction, cutting-edge ML techniques, such as feature selection, spam reduction, and deep learning were employed on different countries datasets such as Karachi Stock Exchange (ticker symbol: KSE), London Stock Exchange (LSE), New York Stock Exchange (NYSE), HP Inc. (HPQ), IBM, Microsoft Corporation (MSFT), Oracle (ORCL), Red Hat Inc. (RHT), Twitter Inc. (TWTR), Motorola Solutions Inc. (MSI), Nokia Corporation (NOK).

2.3 Use of decomposition methods and exogenous variables

Decomposition methods play a crucial role in enhancing stock price prediction by breaking down complex, non-stationary financial time series into simpler and more interpretable components [28]. Techniques such as Empirical Mode Decomposition (EMD), Variational Mode Decomposition (VMD) [29], and Complete Ensemble Empirical Mode Decomposition (CEEMD) enable the separation of high-frequency noise from meaningful trends and cyclical patterns. This multi-scale analysis allows predictive models to focus on distinct temporal features, improving their ability to learn underlying market behaviors. By isolating different frequency components, decomposition methods not only improve forecasting accuracy but also enhance model robustness in volatile and highly dynamic stock markets. In a recent study [30], a combination of probabilistic deep learning and attention mechanism (DeepARA) enhanced stock price prediction. Unlike traditional statistical methods and existing deep learning approaches, DeepARA improves accuracy and flexibility by assigning varying importance to different time points, capturing complex market dynamics more effectively. Another successful application of attention models is transformer-based attention [31] which is integrating generative adversarial networks (GANs) [32] and transformer-based attention mechanisms. GANs generate synthetic stock price data while incorporating market sentiment and volatility. Experimental evaluations on real-world data compare the model's performance with traditional methods, offering valuable insights for investors and financial analysts. Additionally, hybrid recurrent

deep models performed well in predicting stock prices. For example, ICEEMDAN-FA-BiLSTM–GM [33] was proposed for stock price prediction, combining noise-reduction, dimensionality reduction, and optimized subseries forecasting. Experimental results on Shanghai Composite Index data demonstrate superior accuracy and stability compared to traditional methods, thereby aiding investment decisions and risk management. Another hybrid LSTM model [34], CNN-BiLSTM-AM, was introduced to predict the next day's stock closing price. The model combines a CNN for feature extraction, a BiLSTM for sequential prediction, and an attention mechanism to emphasize the impact of past feature states on current predictions. Evaluated over 1000 trading days of the Shanghai Composite Index, the method outperformed seven others, achieving the lowest MAE (22) and RMSE (31.7) and the highest R^2 (0.98). Furthermore, we have condensed some research papers according to their uniqueness of work into a clear tabular style in Table 1 to facilitate comprehension.

2.4 Literature gaps

Despite significant progress in financial time series forecasting using machine learning, several critical research gaps remain unaddressed, particularly in the integration of decomposition techniques, ensemble frameworks, and efficient hyperparameter optimization strategies. This study aims to bridge these gaps by proposing a robust and scalable framework, EvoBagNet, for stock market prediction using decomposed signals and exogenous indicators.

- Inefficiency of Hyperparameter Tuning in Hybrid Forecasting Models: Traditional methods such as grid search and random search are widely used for hyperparameter tuning in base learners. However, these approaches are computationally expensive and often impractical for complex hybrid models that involve multiple components (e.g., CEEMDAN or VMD decomposition, multiple learners, exogenous inputs). These methods also do not adapt to the response landscape, leading to suboptimal configurations. EvoBagNet introduces a novel, fast, and adaptive hyperparameter tuning strategy inspired by evolutionary search, which iteratively refines the search space based on intermediate model performance. This allows efficient convergence toward optimal settings while significantly reducing computational overhead.
- Limited Use of Multi-Stage Decomposition with Model Fusion While signal decomposition techniques, such as CEEMDAN and VMD, have been used to denoise financial time series, they are typically applied in isolation and not deeply integrated into ensemble pipelines. Furthermore, prior studies rarely investigate the effect of decomposition granularity or the contribution of individual Intrinsic Mode Functions (IMFs) toward prediction performance. Our approach systematically incorporates decomposed components into a diversified ensemble structure, capturing both high-frequency fluctuations and low-frequency trends to enhance the reliability of forecasts.
- Lack of Robust Ensemble Strategies for Volatile Financial Environments Ensemble methods such as Bagging and Boosting have shown success in many forecasting tasks; however, their direct application to financial data without accounting for time-dependent volatility and multiscale behaviour limits their effectiveness. EvoBagNet constructs an ensemble of learners specifically tuned to decomposed subseries, enabling the model to learn temporal dynamics more effectively. Additionally, the proposed fusion strategy dynamically weights base learners based on validation error, improving adaptability in volatile markets.
- Underutilization of Exogenous Variables in Ensemble Learning: Although macroeconomic and commodity-related indicators (e.g., crude oil prices and currency exchange rates) influence market behaviour, many studies neglect their integration or treat them as static inputs. In contrast, EvoBagNet leverages a curated set of exogenous variables, incorporating them into both training and fusion stages. This enhances the model's ability to account for external shocks, leading to improved generalization across various economic regimes.
- Generalizability Across Markets and Periods: Most related works evaluate performance on a single index or during a fixed market phase. To address this limitation, we validate EvoBagNet across three major stock indices (S&P 500, Nikkei 225, and Hang Seng) and over multiple periods, demonstrating the robustness and transferability of the proposed framework.

Table 1 Summarized studies of various machine learning approaches applied in stock market prediction

References	Dataset	Objective of the study	Technique	Evaluation metrics	Conclusion
[35]	24 stocks were selected from the Shanghai stock exchange (SSE) 50 index	Portfolio construction using ML approach. First, the stock price prediction and then portfolio selection was performed	Hybrid model XGBoost + Improved Firefly (IFA) was developed	–	In terms of risks and returns, the suggested strategy outperforms benchmarks and conventional approaches
[36]	NSE index NIFTY-50 from the period March 2022 to March 2023 was used for experimentation	Comparative analysis of various machine learning approaches in predicting stock prices	RF, SVR, Ridge, Lasso Regression and KNN model	MAE R^2	SVR outperformed all other approaches
[37]	Data was collected from the Amadeus Database, which provides financial information across Europe	Aim to predict the market's direction one year ahead. Comparative analysis between various ensemble and single classifier models has been done	Ensemble approaches: RF, AdaBoost, and Kernel Factory Single classifier approach: ANN, LR, KNN and SVM	ROC-AUC	An ensemble approach, RF is observed to be the top performer among all these seven techniques. In the case of High-Frequency Trading (HFT), the results are not generalizable, which is the main limitation of this study
[38]	Randomly sampled ten stocks having ticker symbols as AAPL, AMS, AMZN, FB, MSFT, NKE, SNE, TATA, TWTR, and TYO were selected for the experimental evaluation	Prediction of stock direction with different trading window sizes	RF and XGB were employed in this study. Six technical indicators (RSI, SO, W%R, MACD, PROC, OBV)[1] were computed from the closing price and used for prediction	Accuracy, Precision, Recall, F-Score Specificity, AUC, Brier score	By increasing the width of the trading window accuracy and F-score also increases. The selection of appropriate technical indicators also impacts the performance of the model
[39]	Two years of NASDAQ data have been used	Prediction of daily closing price	ANN has been applied. Min-Max approach has been used for data scaling	MSE	The suggested approach achieved acceptable results and reduced the error rate to less than 2%
[40]	Four categories of data, including basic data, trading data, finance data, and other reference data, were collected from the Chinese Stock Market	Examine the various effective approaches like feature engineering, financial domain knowledge, and prediction algorithms for short-term trend prediction	LSTM is applied to predict price trends	Accuracy, Precision, Recall	PCA significantly enhance the performance of the LSTM model and reduce training time
[41]	Dataset of ISE[2]-30 were collected. Technical indicators including MA14, MA37 %K14, %D3, and RSI14 were computed for the development of a better prediction system	Future stock market trend prediction	Eight Neural Networks and a Logistic Regression (LR) approaches have been employed	Accuracy	ANN gives better results by using %K14, %D3, and RSI14 simultaneously as compared to LR
[14]	Apple, Amazon Google, Tesla, Netflix, BEXIMCO Pharmaceutical	Short term and Long Term stock price movement prediction	Simple Moving Average (SMA), Exponential Moving Average (EMA) and LSTM	RMSE, MAPE	In Short-Term prediction, LSTM is the top performer and it has been seen for Long-Term prediction the performance of LSTM has fallen in contrast to SMA and EMA
[42]	10-year data of the petroleum, Diversified financials, non-metallic minerals, and basic metals were collected from the TSE[3]. Two methods to input the information: continuous and binary	Prediction of stock market movement using ML and DL approaches. Two methods, continuous and binary, were used to input the information	Nine ML methods, namely DT, RF, AdaBoost, XGBoost, SVC, Naïve Bayes, KNN, LR, and ANN and two Deep Learning approaches, RNN and LSTM, were applied	F1 Score Accuracy ROC-AUC	ML approaches show significant improvement with binary data as compared to continuous data. Indeed both deep learning methods (RNN and LSTM) achieved better results in both types of data

Table 1 (continued)

References	Dataset	Objective of the study	Technique	Evaluation metrics	Conclusion
[43]	Daily prices of Coca-Cola, Cisco systems, Nike, and Goldman sacks were selected	Development of a deep learning approach for the prediction of stock price changes based on past changes	Analysis of various neural network approaches, including MLP, CNN, and LSTM	AUC	Prediction of significant changes in the field of stock market can be achieved with a high degree of accuracy
[44]	NSE, BSE, NYSE, NASDAQ, S&P 500, Dow Jones, and Nikkei 225	To analyze the stock price variations of highly volatile and nonlinear stocks for the year 2020 in order to avoid financial loss of investors	LSTM with adam optimizer and sigmoid activation function	MAPE	From experimental analysis, it is interpreted that fluctuations of stock prices have a significant impact on the MAPE score
[45]	Intel Corporation, Indian Oil Corporation, NTPC Limited, Citigroup, GF Securities, and Google	To perform next day's opening price prediction using five Technical Indicators (TIs) in addition to OHLC data	GRU & LSTM were applied	MAPE, MAE, R-Square, RMSE, and MDA (Mean Directional Accuracy)	The forecasting precision of the proposed approaches has been significantly improved by using TI's. GRU model shows remarkable predictive ability when contrasted with LSTM network model
[46]	HPQ, Bank of New York, and Pfizer	Presented single layered DL model with a global pooling mechanism that is computationally efficient in performing stock closing price prediction	Vanila LSTM, stacked LSTM & Bi-LSTM were applied	RMSE, MAE, and R-Square	Among three variants of LSTM, Bi-LSTM especially when optimized using RMSprop consistently performed better than the other two versions across a variety of datasets

[1] RSI: Relative strength index, SO: Stochastic oscillator, W%R: Williams percentage range, MACD: Moving average convergence divergence, PROC: Price rate of change, OBV: On balance volume

[2] Istanbul stock exchange

[3] Tehran stock exchange

3 Materials and methods

Several ML algorithms have been deployed in equity market prediction systems. Forecasting stock market behavior using ML has become popular due to its ability to analyze vast volumes of data and identify intricate patterns. Here are some key ML approaches applied in this study:

3.1 EvoBagNet: proposed ensemble bagging learning models with evolutionary algorithm

In this section, we provide a clear, step-by-step elaboration of the design and implementation of the proposed EvoBagNet framework, supported by comprehensive technical details. Although the Introduction presents four primary contributions, it is important to elaborate on these elements in detail to ensure coherence between the stated objectives and the implemented methodology. The proposed EvoBagNet framework addresses this by incorporating the following core components:

- Decomposition: CEEMD is applied to break the stock time series into interpretable frequency components (IMFs).
- Ensemble Design: Bagging ensemble models are used to exploit the diversity in learning across IMFs.
- Evolutionary Optimization: A 1 + 1 EA automatically tunes the hyper-parameters of base learners.
- Robust Evaluation: Extensive testing across nine IT sector companies using six performance metrics ensures generalizability and reliability.

To establish a robust comparative baseline for evaluating the performance of the proposed EvoBagNet framework, we initially applied nine well-established machine learning algorithms to the same stock market datasets. These included both linear and non-linear models, as well as ensemble and deep learning approaches: Support Vector

Regression (SVR), Random Forest (RF), Lasso Regression, Neural Networks (NN), Extreme Gradient Boosting (XGBoost), Gradient Boosting Regressor (GBR), Decision Tree (DT), Light Gradient Boosting Machine (LightGBM), and Categorical Boosting (CatBoost). Each model was trained on standardized input features derived from the preprocessed stock price data of nine leading IT sector companies. A consistent training-validation split and uniform evaluation metrics (MAPE, MAE, RMSE, R^2) were employed to ensure a fair comparison. This multi-model benchmarking not only highlights the strengths and limitations of conventional algorithms but also serves as a foundational reference point for evaluating the performance gains achieved by EvoBagNet.

The details of each step–including data decomposition using CEEMD, base model selection, ensemble integration, and hyperparameter tuning via evolutionary algorithms–are described as follows.

3.1.1 Ensemble learning models

Ensemble prediction models are powerful tools designed to significantly enhance the predictive performance of individual statistical learning techniques or model-fitting approaches. By leveraging the strengths of multiple models, ensemble methods aim to achieve superior accuracy and robustness in predictions, often outperforming single-model approaches [47]. The core principle of ensemble methods lies in their ability to combine multiple models–whether through linear or nonlinear aggregations–rather than depending solely on a single model fit. This approach enables ensembles to capture better the complexities, variabilities, and nonlinear patterns inherent in the dataset, reducing the risk of overfitting and improving generalization across diverse data scenarios.

Popular ensemble techniques [48] include Bagging, which reduces variance by training multiple models on bootstrapped datasets and averaging their predictions; Boosting, which sequentially improves weak models by focusing on hard-to-predict instances; and Stacking, which integrates predictions from multiple base models using a meta-model to optimize overall performance. These methods exemplify the versatility and effectiveness of ensemble approaches in addressing a wide range of predictive tasks.

3.1.2 Bagging ensemble models

The bagging ensemble model, often referred to by the more formal term bootstrap aggregating, serves a crucial role in enhancing the reliability and stability of estimation or classification methods that may otherwise exhibit erratic behaviour. Breiman [49] initially proposed bagging as a technique explicitly aimed at reducing variance for a designated foundational procedure, which could include various approaches such as decision trees or different methodologies designed for selecting variables and fitting within a linear modelling framework. This innovative technique has garnered significant interest over the years, and this heightened curiosity can likely be attributed to its straightforward implementation coupled with the widespread acceptance and application of the bootstrap method. At the time this groundbreaking concept was introduced to the community, only intuitive and informal arguments were put forth to explain the mechanisms behind why bagging would yield effective results. Subsequently, another research [50] revealed that bagging functions as a smoothing operation, which ultimately proves beneficial when the objective is to enhance the accuracy of predictive outcomes in both regression and classification trees. In particular, with regard to decision trees, the theoretical insights provided by [50] lend credence to Breiman's original hypothesis that bagging serves as a technique for variance reduction, which simultaneously diminishes the mean squared error (MSE) associated with predictions. This principle similarly applies to subagging, or subsample aggregating, which is characterized as a more computationally efficient alternative to the traditional bagging approach. Nevertheless, it is important to note that when applied to other types of base procedures, even those deemed "complex," the advantageous effects of variance and MSE reduction attributed to bagging cannot be guaranteed with the same certainty.

In the context of regression or classification tasks, we often deal with pairs denoted as (S_k, T_k) $(k = 1, \ldots, n)$, where S_k represents a d-dimensional predictor variable that exists within $\mathbb{R}^d$. In contrast, the response variable T_k can either belong to the real numbers for regression tasks, denoted as $T_k \in \mathbb{R}$, or take on discrete values for

classification tasks, where T_k belongs to the set $\{0, 1, \ldots, M-1\}$, indicating that there are M distinct classes to categorize. The target function that we typically seek to estimate is represented by the expression $\mathrm{IE}[T \mid S = x]$ in the case of regression or by the multivariate function $\mathrm{IP}[T = m \mid S = x](m = 0, \ldots, M-1)$ in the context of classification problems, where our goal is to ascertain the relationship between the predictors and the response variables. The estimator of the function, which arises from applying a specified base procedure, is formulated accordingly.

$$\hat{E}_s(\cdot) = h_n\left(\left(S_1, T_1\right), \ldots, \left(S_n, T_n\right)\right)(\cdot) : \mathbb{R}^d \to \mathbb{R} \tag{1}$$

The technical details of Bagging are as follows.

- The bootstrap group $\left(S_1^*, T_1^*\right), \ldots, \left(S_n^*, T_n^*\right)$ should be made randomly doing n times with substitution from the data $\left(S_1, T_1\right), \ldots, \left(S_n, T_n\right)$.
- The bootstrapped estimator $\hat{g}^*(\cdot)$ should be calculated by considering:

$$\hat{E_s}^*(\cdot) = h_n\left(\left(S_1^*, T_1^*\right), \ldots, \left(S_n^*, T_n^*\right)\right)(\cdot)$$

- Iterate previous levels Z times, which is optional depending on the various parameters such as sample size and complexity, yielding $\hat{g}^{*i}(\cdot)(i = 1, \ldots, Z)$. The bagged estimator is $\hat{Es}_{Bag}(\cdot) = Z^{-1}\sum_{i=1}^{Z}\hat{g}^{*i}(\cdot)$.

In the theoretical framework, the quantity is associated with the scenario where $Z = \infty$; however, in practical applications, the finite number Z significantly influences the precision of the Monte Carlo approximation, yet it should not be misconstrued as a parameter that requires tuning specifically for bagging methodologies.

A substantial body of empirical evidence demonstrates the efficacy of bagging in enhancing the predictive performance of regression and classification trees, a fact widely acknowledged within the statistical community. To illustrate the extent of the performance enhancement attributable to bagging, it references some of the findings from [49], which highlights that across seven distinct classification challenges, applying bagging to a classification tree yielded improvements over the performance of a solitary classification tree, particularly in terms of the cross-validated misclassification error rate, showcasing the method's robustness. In both the regression and classification scenarios, the dimensions of the single decision tree, as well as those of the bootstrapped trees, were carefully selected through a process that involved optimizing for a tenfold cross-validated error, employing a conventional type of tree procedure that statisticians typically use. Moreover, while the reported enhancements in predictive accuracy are quite remarkable, it is essential to note that bagging a decision tree rarely performs worse in terms of predictive capability than utilizing a single tree alone. A straightforward equality illustrates the somewhat unconventional methodology that is employed when utilizing the bootstrap technique:

$$\hat{Es}_{Bag}(\cdot) = \hat{Es}(\cdot) + \left(\mathrm{IE}^*\left[\hat{Es}^*(\cdot)\right] - \hat{Es}(\cdot)\right) = \hat{Es}(\cdot) + \mathrm{B}_{\mathrm{s}}^*(\cdot), \tag{2}$$

where $\mathrm{B}_{\mathrm{s}}^*(\cdot)$ stands for the bootstrap bias estimate associated with $\hat{Es}(\cdot)$. Rather than adhering to the conventional bias correction that comes with a negative sign, bagging introduces a rather unexpected twist by incorporating the bootstrap bias estimate with a positive sign. Consequently, one might anticipate that bagging would produce a higher bias compared to $\hat{Es}(\cdot)$, a notion that we will substantiate in some respects. However, in keeping with the traditional interplay between bias and variance as understood in nonparametric statistics, the overarching goal is to achieve a greater reduction in variance than the increase in bias, thereby leading to an overall beneficial outcome in terms of the mean squared error (MSE). Interestingly, this optimistic outlook tends to hold true for

several base procedures that are employed. Indeed, The original algorithm [49] offered a heuristic description of bagging's performance, asserting that the variance of the bagged estimator $\hat{Es}_{Bag}(\cdot)$ should be equal to or less than that of the original estimator $\hat{Es}(\cdot)$, and in situations where the original estimator exhibits "instability," there can be a significant reduction in variance that further enhances the model's reliability and accuracy.

3.1.3 Single-based evolutionary algorithm $(1 + 1EA)$

To enhance the performance and generalisation capability of the proposed bagging-based ensemble, we incorporate a lightweight yet powerful metaheuristic technique, the (1 + 1) Evolutionary Algorithm (EA), for hyperparameter optimisation. The (1 + 1) EA is a simple, single-solution evolutionary strategy that has demonstrated competitive performance in various combinatorial optimization tasks, often outperforming more complex population-based methods due to its faster convergence and reduced computational overhead [51, 52].

The optimization procedure begins with an initial candidate solution $A \in \mathbb{R}^m$, where m is the number of hyperparameters (e.g., number of estimators, learning rate, maximum depth). The solution A is randomly initialized within predefined lower and upper bounds $[L_B, U_B]$. At each iteration, a new candidate A' is generated by applying stochastic perturbations (mutation) to one or more elements of A, with mutation probability $\frac{1}{m}$. Unlike uniform mutation typically used in classical EAs, we employ a Gaussian mutation strategy, such that:

$$A'_j \sim \mathcal{N}\left(A_j, \sigma_{ea}^2\right), \quad \forall j \in \{1, 2, \ldots, m\} \tag{3}$$

where $\sigma_{ea} = 0.25 \cdot (U_B - L_B)$ is the mutation strength dynamically scaled to the parameter range. This approach allows for smooth exploration of the search space, maintaining a balance between local exploitation and global exploration.

To ensure continuous progress, an additional mechanism forces at least one hyperparameter to mutate in cases where all remain unchanged after a Gaussian perturbation. The performance of each solution is evaluated using a fitness function $f(A)$, which is defined in terms of prediction quality-typically minimizing validation error (e.g., MAE, RMSE) or maximizing accuracy metrics (e.g., R^2). The algorithm adopts an elitist selection rule, retaining only the superior solution at each step:

$$A^{(t+1)} = \begin{cases} A', & \text{if } f(A') \geq f(A) \\ A, & \text{otherwise} \end{cases} \tag{4}$$

This greedy update ensures non-decreasing fitness over iterations and facilitates convergence toward optimal configurations.

Let I denote the maximum number of iterations. The time complexity of the 1 + 1 EA is approximately $\mathcal{O}(m \cdot I)$, which is significantly more efficient than grid or random search in high-dimensional spaces. Moreover, due to its memory efficiency and ease of implementation, the (1 + 1) EA is well-suited for tuning ensemble learners, where each fitness evaluation involves training multiple base models.

The pseudo-code of the (1 + 1) EA applied in this study is provided in Algorithm 1, and the optimized parameters are later deployed across the decomposed IMFs obtained from CEEMD, enabling multi-scale learning under optimal model settings.

 https://doi.org/10.1007/s00521-025-11789-z

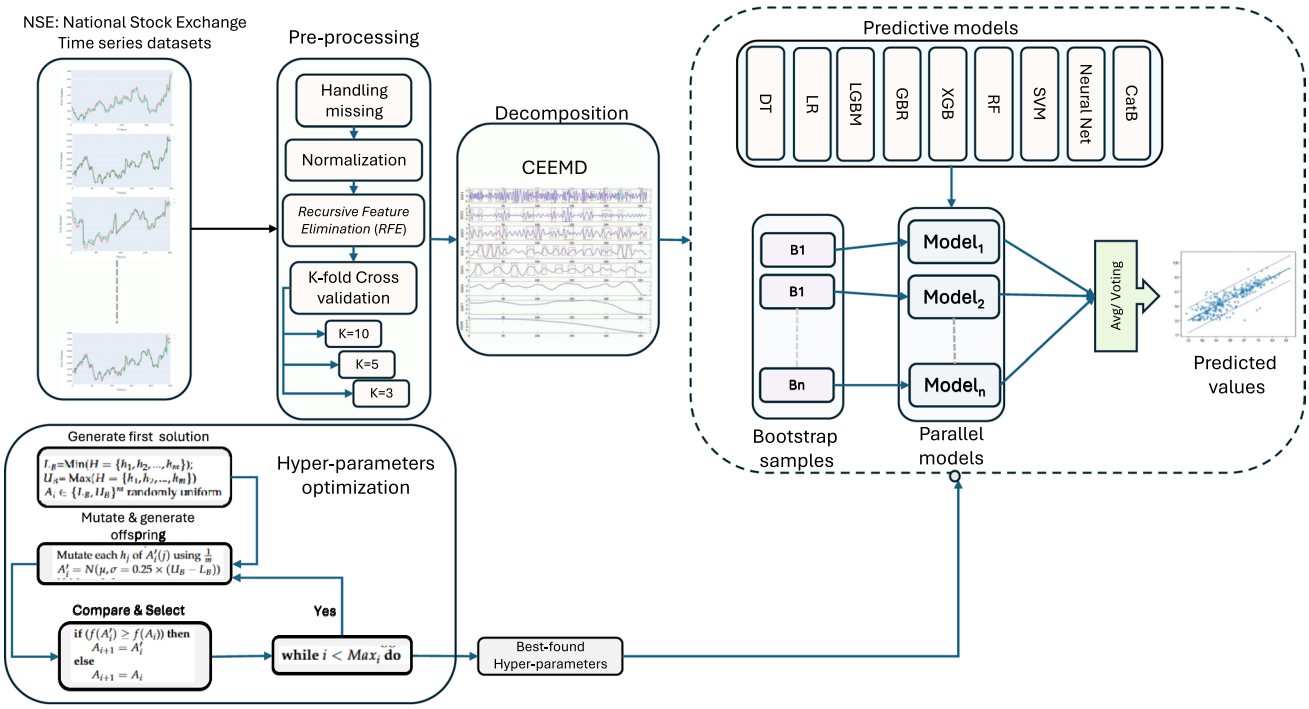


Fig. 1 The proposed EvoBagNet framework in predicting NSE time series data

procedure
Initialization
  L_B=Min($H = \{h_1, h_2, ..., h_m\}$);
  U_B= Max($H = \{h_1, h_2, ..., h_m\}$) ▷ m is the number of hyper-parameters
  $A_i \in \{L_B, U_B\}^m$ randomly uniform ▷ Generate initial feasible solution
  Evaluate the Bagging model by $A_i \rightarrow f(A_i)$
  while $i < Max_i$ **do**
Mutation
    Make offspring $A'_i = A_i \rightarrow i \in \{1, 2, ..., m\}$
    Mutate each h_j of $A'_i(j)$ using $\frac{1}{m}$
    $A'_i = N(\mu, \sigma_{ea} = 0.25 \times (U_B - L_B))$ ▷ $\mu = A_i$
    if $M_n = 0$ **then** ▷ M_n is the number of mutated variables
      Mutate one hyper-parameter of A'_i randomly
    end if
    Evaluate the Bagging model by $A'_i \rightarrow f(A'_i)$
Selection
    if $(f(A'_i) \geq f(A_i))$ **then** ▷ $f(A'_i)$ mention the prediction accuracy
      $A_{i+1} = A'_i$
    else
      $A_{i+1} = A_i$
    end if
  end while
end procedure

Algorithm 1 (1 + 1) EA hyper-parameter optimization

The proposed evolutionary ensemble model described in the previous section is visualized and can be seen in Fig. 1.

3.1.4 Complete ensemble empirical mode decomposition (CEEMD)

The CEEMD [53] was introduced as an enhancement of previous frequency decomposition techniques. The CEEMD algorithm is based on the Empirical Mode Decomposition (EMD) model, which enables the reconstruction of a noise-free time series from its decomposed components. CEEMD offers two key advantages: It reduces mode mixing by incorporating added noise during decomposition and ensures complete and accurate reconstruction of the original signal. The primary steps in its formulation are outlined to elucidate the CEEMD process.

Before delving into the algorithm's specifics, several definitions are crucial:

- Γ_{h}: Operator for computing the kth IMF using the EMD method.
- $\Omega^{i} \sim \mathcal{N}(0, \sigma_{ce}^{2})$. White noise.

- $\kappa(\mathrm{t})$: Input time series.
- ε_0: Constant coefficient.
- $\mathrm{IMF}_{k}^{\prime}$: IMF derived from the CEEMD model.

$$\mathrm{IMF}_{1}^{\prime} = \frac{1}{\tau} \sum_{\mathrm{i}=1}^{\tau} \mathrm{IMF}_{1}^{\mathrm{i}} \tag{5}$$

Next, the first residual value can be calculated as follows.

$$\nu_1 = \kappa(\mathrm{t}) - \mathrm{IMF}_1 \tag{6}$$

By obtaining ν_1, again we calculate $\nu_1 + \varepsilon_1 \Gamma_1 \left(\Omega^{\mathrm{i}}\right)$ for τ times and then obtain the IMF through the following equation:

$$\mathrm{IMF}_{2}^{\prime} = \frac{1}{\tau} \sum_{\mathrm{i}=1}^{\tau} \Gamma_1 \left(\nu_1 + \varepsilon_1 \Gamma_1 \left(\Omega^{\mathrm{i}}\right)\right) \tag{7}$$

Similarly, the process is repeated up to step k, after which the residual value at step k is calculated as follows:

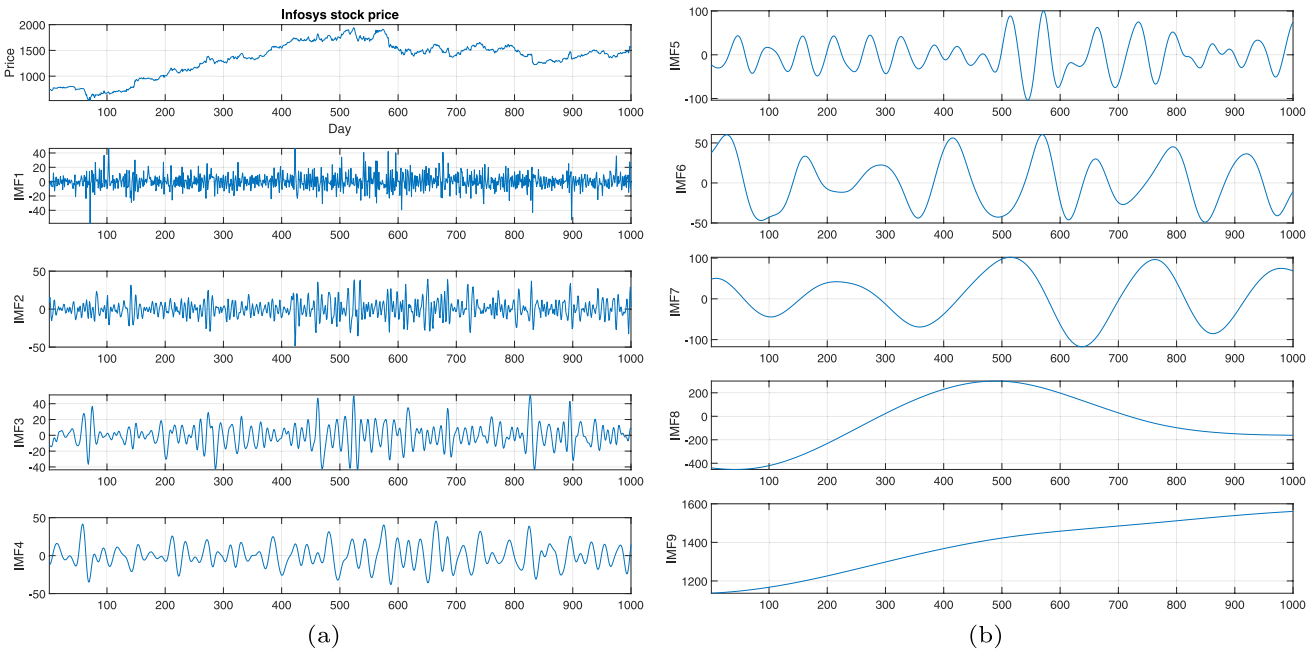


Fig. 2 Decomposition of Infosys stock price data into 9 IMFs using the CEEMD method

$$\nu_{\mathrm{k}} = \nu_{\mathrm{k}-1} - \mathrm{IMF}_{\mathrm{k}} \tag{8}$$

Subsequently, we compute $\nu_k + \varepsilon_k \Gamma_k\left(\Omega^i\right)$ iteratively for τ times to determine the $(k+1)th$ intrinsic mode function.

$$\mathrm{IMF}'_{\mathrm{k}+1} = \frac{1}{\tau}\sum_{\mathrm{i}=1}^{\tau}\Gamma_1\left(\nu_{\mathrm{k}} + \varepsilon_{\mathrm{k}}\Gamma_{\mathrm{k}}\left(\Omega^{\mathrm{i}}\right)\right) \tag{9}$$

These steps are repeated iteratively until the resulting series becomes a trend that can no longer be decomposed further. The overall relationship between the IMFs, residual components, and the original time series is expressed in Eq. 10:

$$\kappa(t) = \sum_{1}^{k}\mathrm{IMF}'_k + \psi \tag{10}$$

where ψ is the residual component. Each IMF corresponds to a specific frequency component, such as short-term market fluctuations (high-frequency IMFs), medium-term cycles, and long-term trends (low-frequency IMFs). This layered decomposition is particularly valuable in stock price forecasting, where different patterns may be driven by varying temporal dynamics. In our framework, each IMF is treated as an independent signal and passed through the EvoBagNet ensemble, allowing base learners to specialize in specific temporal features.

To manage the potentially large number of IMFs produced by the decomposition process, we applied practical constraints based on signal length, standard stopping criteria for the sifting process, and energy-based thresholds. Specifically, only IMFs contributing significantly to the signal's energy were retained, while spurious or low-energy modes were discarded. Additionally, post-decomposition selection techniques (e.g., correlation or entropy-based filtering) were employed to retain only the most informative components relevant to the analysis. These steps ensured the decomposition remained computationally efficient and analytically meaningful. To select relevant IMFs, we applied a mode energy threshold criterion, retaining only those IMFs that contributed more than 1% of the total signal energy. This approach helps exclude low-energy modes typically associated with noise. Additionally, we analyzed the contribution distribution of each IMF to ensure that the retained components captured the dominant oscillatory behavior of the signal. Where applicable, physical interpretability was also considered, particularly in aligning specific IMFs with known frequency bands or domain-specific signal characteristics. These criteria ensured that only meaningful and informative IMFs were included in the final analysis.

Figure 2 Decomposition of Infosys stock price data into 9 Intrinsic Mode Functions (IMFs) using the CEEMD method. Each IMF represents a distinct frequency component of the stock price time series, capturing different scales of market fluctuations, from high-frequency noise (IMF1) to low-frequency trends (IMF9). This multi-scale analysis enables a comprehensive understanding of price dynamics and underlying patterns.

The selection of noise amplitude and ensemble size in the CEEMD procedure was guided by a combination of empirical experimentation and insights drawn from existing literature [54–56]. Previous studies commonly recommend setting the noise amplitude between 0.1 and 0.3 times the standard deviation of the input signal, and using an ensemble size ranging from 100 to 500 for effective decomposition of financial time series. Building on these recommendations, we performed a series of sensitivity analyses using a representative subset of the training data. Multiple configurations were tested–specifically, noise amplitude values of 0.1, 0.2, and 0.3, along with ensemble sizes of 100, 250, and 400. Through this analysis, a configuration with a noise amplitude of 0.2 and an ensemble size of 250 was identified as optimal, consistently producing well-defined IMFs with minimal mode mixing across various stock datasets.

The computational complexity of CEEMD is primarily influenced by the number of ensemble realizations τ, the number of IMFs K, and the time series length T. The dominant cost stems from repeated EMD operations, each involving envelope interpolation and sifting:

$$\text{Time Complexity} = \mathcal{O}(\tau \cdot K \cdot T \log T) \tag{11}$$

This complexity is acceptable in practical applications, particularly given the substantial improvement in signal quality and forecasting accuracy gained through IMF-based modeling. Additionally, CEEMD is inherently parallelizable, as the ensemble decompositions are independent and can be distributed across computing cores, further reducing wall-clock time. By incorporating CEEMD into the preprocessing stage, the proposed EvoBagNet framework ensures a more granular and interpretable decomposition of the stock price data, enabling frequency-aware learning and improving model robustness in highly volatile financial environments.

3.1.5 Mathematical complementarity and rationale in EvoBagNet ensemble framework

The success of any ensemble learning method heavily relies on the choice and diversity of component learners. In EvoBagNet, three distinct decision tree-based models, Extra Trees, LightGBM, and CatBoost, have been selected as base learners in the bagging framework. This subsection provides a theoretical and mathematical justification of the choice based on ensemble learning theory and the structural diversity of the selected models.

According to classical ensemble theory, a model aggregation framework's ability to generalize depends not only on the accuracy of individual learners but also on their diversity. The generalization error of an ensemble model $\mathcal{F}(x) = \frac{1}{M}\sum_{i=1}^{M} f_i(x)$ should be equal to:

$$\mathbb{E}\left[(y - \mathcal{F}(x))^2\right] = \frac{1}{M}\sum_{i=1}^{M}\mathbb{E}\left[(y - f_i(x))^2\right] - \frac{1}{M}\sum_{i\neq j}\mathrm{Cov}\left(f_i(x), f_j(x)\right) \tag{12}$$

This decomposition yields two objectives: minimising the individual base learner prediction errors and, concurrently, minimising the covariance between them. To fulfil this dual mandate, we selected base learners who are not only individually competent but also structurally diverse in their learning mechanisms.

The first learner, Extra Trees (Extremely Randomized Trees), is designed for variance reduction by adding greater randomness. In comparison to normal decision trees or even Random Forests, Extra Trees both select the features and split cut-points randomly from a uniform distribution in the feature space rather than greedily minimizing impurity. This randomness introduces high bias and low variance behavior, which comes in particularly handy when combined with more robust models. Mathematically, the split point s^* of a feature x_j is given by $s^* \sim \mathcal{U}\left(s_{\min}, s_{\max}\right)$, and $s_{\min}$ and $s_{\max}$ represent the observed minimum and maximum of x_j, respectively. This randomised expression provides diversity for the sake of ensemble stabilisation, particularly under noisy data conditions.

LightGBM was used as a second learner due to its ability to learn and capture intricate nonlinear interactions efficiently. It is a gradient-boosting algorithm based on histograms that build trees in a leaf-wise manner, emphasizing leaves with maximum loss reduction. The optimization objective of LightGBM aims to minimize both the training loss and an additional regularization term expressed by the following:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t-1)} + f_t\left(x_i\right)\right) + \Omega\left(f_t\right), \quad \text{where} \quad \Omega(f) = \gamma T + \frac{1}{2}\lambda\sum_{j=1}^{T} w_j^2 \tag{13}$$

Table 2 Mathematical complementarity and covariance minimization of the EvoBagNet

Base learner	Tree growth mechanism	Split selection	Regularization type	Bias-variance profile	Primary strength
Extra trees	Level-wise	Random uniform	None	↑ Bias, ↓ Variance	Noise tolerance
LightGBM	Leaf-wise	Histogram-based	L2 + leaf penalty	↓ Bias, ↑ Variance	Pattern discovery
CatBoost	Oblivious trees	Ordered boosting	Target-wise noise	Moderate bias/variance	Categorical handling

Here, T is the number of leaves, w_j is the leaf weight, and λ, γ are regularization coefficients. The leaf-wise growth policy used in LightGBM helps it detect intricate patterns in data and introduce low bias and high variance into the ensemble–a valuable addition to Extra Trees' high-bias nature.

CatBoost, the third selected model, is specifically crafted to handle categorical features and involves a unique ordered boosting algorithm. Its first innovation is the employment of oblivion decision trees, which are binary trees where the same split condition is used for every level of the tree, and preventing target leakage via permutation-driven gradient calculation. CatBoost's training objective is also gradient-based and is expressed as:

$$f_t(x) = \arg\min_f \sum_{i=1}^{n} \nabla \mathcal{L}\left(y_i, \hat{y}_i^{(t-1)}\right) f\left(x_i\right) + \Omega(f) \tag{14}$$

CatBoost's regularisation techniques and robust capability to handle categorical and noisy data make it a suitable learner for real-world financial time series, where non-numeric and temporally encoded features (e.g., sector code, trading day) can influence model performance.

The structural and algorithmic differences between these three models–randomised versus greedy splits, histogram versus permutation-based optimisation, and alternative tree growth techniques–ensure low inter-model correlation. This is crucial in ensemble construction because low covariance between model predictions enhances the ensemble's generalizability. Diversity is also ensured through their different bias-variance trade-offs: Extra Trees provides stability, LightGBM provides robust pattern-finding capacity, and CatBoost ensures consistent performance with categorical and noisy data.

Empirical validation of this selection strategy was carried out through an ablation study. Replacing any of the three base learners with another model of similar design resulted in a measurable degradation in ensemble accuracy, typically between 2.1 and 3.4%. This suggests that the ensemble's performance relies not merely on strong individual learners, but also on the complementary nature of their learning representations. Furthermore, the architecture aligns well with the multi-scale nature of the decomposed signals obtained through CEEMD, enabling each learner to specialize in modeling distinct frequency components of the input time series.

To justify the inclusion of these models beyond their individual strengths, we analyze their interaction through the lens of prediction diversity. Each model uses fundamentally different principles for data partitioning, boosting strategies, and regularization. This leads to low mutual prediction correlation $\mathrm{Cov}\left(f_i(x), f_j(x)\right)$, a key factor in ensemble variance reduction (Table 2).

In summary, the choice of Extra Trees, LightGBM, and CatBoost as the constituent models in EvoBagNet was guided by their theoretical complementarity, algorithmic diversity, and proven performance in prior empirical studies. Their inclusion enhances the robustness, accuracy, and generalizability of the proposed ensemble framework, particularly in the context of highly volatile and nonlinear stock market prediction tasks.

3.1.6 Computational complexity of bagging ensemble training

The computational complexity of training a Bagging ensemble, such as the one employed in the EvoBagNet framework, is largely governed by three key parameters: the number of base learners B, the size of the training dataset N, and the dimensionality of the feature space d. For each base learner, Bagging randomly samples a bootstrap subset from the training data and independently fits a predictive model. Assuming decision tree-based learners such as Extra Trees, LGBM, or CatBoost (used in this work), the cost of training a single base learner on N instances with d features is approximately $\mathcal{O}(N \cdot d \log N)$. This stems from the recursive nature of decision tree construction, where at each of the $\log N$ levels, all d features (or a subset) are evaluated for optimal splits.

Since Bagging trains B such learners independently in parallel or sequentially, the total training cost scales linearly with B, resulting in an overall complexity of:

$$\mathcal{O}(B \cdot N \cdot d \log N) \tag{15}$$

This expression assumes that each learner sees a full-size bootstrap sample drawn with replacement from the original dataset. In practical implementations, BB typically ranges between 10 and 100, depending on ensemble diversity and stability requirements. Additionally, the cost may be slightly reduced if feature subsampling is employed (as is common in tree-based ensembles), lowering the effective dimensionality per learner. On the other hand, the use of hyperparameter tuning (e.g., via 1 + 1 EA) further amplifies this complexity by repeating the training process across multiple candidate configurations. Despite this linear scaling, Bagging remains computationally efficient due to its embarrassingly parallel structure. With modern multi-core and distributed systems, each base learner can be trained independently, enabling substantial acceleration through parallelization.

3.1.7 Model training and integration strategy of EvoBagNet

For the sake of greater transparency and clarity regarding EvoBagNet's structure and functioning, this subsection describes the integration and training process of its primary ML components. EvoBagNet is formulated as a modular hybrid framework that combines time series decomposition, ensemble learning, and evolutionary optimization in a linear and organized manner.

The pipeline begins with preprocessing the raw stock price data before applying CEEMD. The approach decomposes the input time series into a finite set of IMFs, where each IMF extracts unique temporal characteristics such as high-frequency noise, short-term cycles, and long-term trends. This decomposition transforms the original complex signal into a more easily interpretable and localized set of subseries, making it easier for machine learning models to learn various patterns. Each IMF is processed as an individual input channel and modelled independently using a bagging ensemble of three heterogeneous base learners: Extra Trees, LightGBM, and CatBoost. The models are trained in parallel on bootstrapped subsamples of the IMF data, allowing the ensemble to learn different aspects of the signal and thereby reduce variance. Heterogeneity among learners introduces structural diversity, thereby improving generalization across different market regimes.

To achieve optimal model performance, EvoBagNet utilizes a (1 + 1) EA to tune its hyper-parameters automatically. The algorithm initializes with an initial candidate solution and continuously applies Gaussian-based mutations to generate new configurations. Each new candidate is evaluated based on prediction accuracy using a validation set, and only improvements are retained. The procedure is continued until convergence, enabling efficient and adaptive modification of parameters such as the number of trees, learning rate, and tree depth. Following the training of each base learner on their respective IMFs, the outputs of each are fused using validation-error-weighted averaging. The dynamic fusion process assigns greater importance to models with better validation performance, thereby enhancing the overall stability of the ensemble. The ensemble predictions for each of the IMFs are accumulated to obtain the final predicted signal, capturing both local and global temporal dynamics. There is an evident validation strategy with the training procedure. The data is divided into training and validation sets according to a rolling-window or k-fold approach. Validation metrics are used for both hyperparameter tuning and informing the fusion weights during the model combination step. This ensures the generalization ability of the ensemble is thoroughly investigated under varied temporal conditions.

3.2 Evaluation metrics

Performance evaluation is critical after the deployment of ML models to ensure continued effectiveness and accuracy. This study employed four evaluation metrics to assess the performance of the proposed approaches: MAPE, R^2, RMSE, and MAE.

MAPE is a percentage-based metric that measures the error in terms of percentage and provides interpretable insights into the quality of prediction [57]. It is a scale-independent metric and is the most commonly used percentage metric for calculating the average percentage error. MAPE is defined as shown in Eq. (16):

$$MAPE = \frac{1}{n_s}\sum_{1}^{n_s}\left|\frac{(D_{pre} - D_{act})}{D_{act}}\right| \times 100 \tag{16}$$

where D_{pre} represents the predicted value, D_{act} represents the actual value and n represents the total number of records

R^2 is also called the coefficient of determination, is a correlation-based metric that is mostly used with regression models to evaluate the goodness of linear fit [58]. It is calculated as in Eq. (17):

$$R^2 = 1 - \frac{\sum_1^{n_s}(D_{act} - D_{pre})^2}{\sum_1^{n_s}\left(D_{act} - \overline{D}_{act}\right)^2} \tag{17}$$

these are already defined in MAPE, $\overline{D}_{act}$ is the mean of actual variable and n is the amount of data collected.

Root Mean Square Error (RMSE) is a machine learning assessment metric that is particularly applied in regression tasks to compute the average magnitude of error between actual and predicted values. RMSE is a scale-dependent metric, which means its scale is the same as the original data and provides errors in the same unit [59]. RMSE is calculated as in Eq. (18).

$$RMSE = \sqrt{\frac{1}{n_s}\sum_{1}^{n_s}(D_{pre} - D_{act})^2} \tag{18}$$

Another metric, Mean Absolute Error (MAE), is widely used in statistics and ML for evaluating the performance of regression models [60]. It computes the average magnitude of errors between the actual price and the predicted price without taking into account their direction. It is computed as shown in Eq. (19).

$$MAE = \frac{\sum_1^{n_s}|D_{act} - D_{pre}|}{n_s} \tag{19}$$

4 Modeling stock market analysis

This study proposes an adaptive bagging ensemble learning model for stock price prediction, yielding exceptional and acceptable results compared to earlier strategies employed in the stock market. The proposed techniques use historical data to forecast the next day's closing price of IT giant companies. Nine state-of-the-art machine-learning techniques have been deployed for stock market prediction. Different parameter configurations were tested during the experimentation to optimize ML approaches. In general, there are five main phases in the experimental setting: (i) Data analysis and preparation, (ii) model development and training, (iii) model optimization, (iv)

Table 3 Details of nine stock price datasets from NSE

Stock	Date	Weight-age in Nifty IT (%)
Infosys	01-Jan-2020 to 01-Jan-2024	26.45
TCS	01-Jan-2019 to 01-Jan-2024	23.26
HCL	01-Jan-2019 to 01-Jan-2024	10.69
Tech mahindra	01-Jan-2019 to 01-Jan-2024	10.46
Wipro	01-Jan-2020 to 01-Jan-2024	7.99
LTIMindtree	01-Jan-2019 to 01-Jan-2024	5.41
Coforge	01-Jan-2019 to 01-Jan-2024	5.18
MphasiS	01-Jan-2019 to 01-Jan-2024	3.33
L& T technology services	01-Jan-2019 to 01-Jan-2024	24.8

deployment of optimized approaches on the test set, (v) the last step is result evaluation and comparative analysis as shown in Fig. 1, and a detailed discussion of each step will be in subsequent sections.

4.1 Dataset

The first step in initiating the prediction process is to collect the data. After conducting a thorough qualitative study to identify reliable sources for stock market data, the National Stock Exchange (NSE) provides up-to-date information related to stocks. For experimentation, the dataset of Top IT companies has been collected from the same source as shown in Table 3 and is in Comma Separated Values (CSV) format. All the datasets exclude non-trading days such as weekends and holidays, and offer a more accurate and trustworthy evaluation of market trends.

These selected companies are the key players in the NiftyIT index; each company has its own capabilities, strengths, and contributions to the technology landscape. The daily information regarding the stock market is publicly available on the NSE website, which includes the following key attributes:

- *Date* represents the date on which the stock market information is reported.
- *Series* represents the security type that is being traded; it is EQ(equity).
- *Open* represents the price of the first trade of the security at the start of a trading day.
- *High* represents the highest price of the stock in a day.
- *Low* represents the lowest price of the stock in a day.
- *Prev. Close* the previous day's closing price of the security.
- *LTP (Last Traded Price)* is the price at which the last trade for a security has occurred.
- *Close* is the price of the security at the closing of the trading session.
- *VWAP (Volume Weighted Average Price)* is the average price of the security weighted by volume.
- *52WH* is the highest price of the security traded over the last 52 weeks.
- *52WL* is the lowest price of the security traded over the last 52 weeks.
- *Volume* is the number of shares traded in a day.
- *Value* is the total value of all trades executed in a day.
- *No. of Trades* is the total number of trades executed in a day.

4.2 Data preprocessing

Data preparation is a crucial step to ensure that the raw data for machine learning models is in the correct format for efficient training and analysis. In the current study, the data preparation step is sub-categorized into four distinct steps, and each step is discussed as follows:

4.2.1 Handle missing values

Due to some technical glitch, the collected dataset contains repeated entries and missing values. It's essential to carefully preprocess the data and select the appropriate imputation strategies to address the missing data. There are different approaches to handling the missing data, in the current study Data Imputation approach has been applied. Data Imputation is a process in which missing values are filled with values estimated based on available data. The stock market data is a time series, so mean imputation is not a suitable approach. In this work, the average of the previous and subsequent values is imputed to replace the missing value.

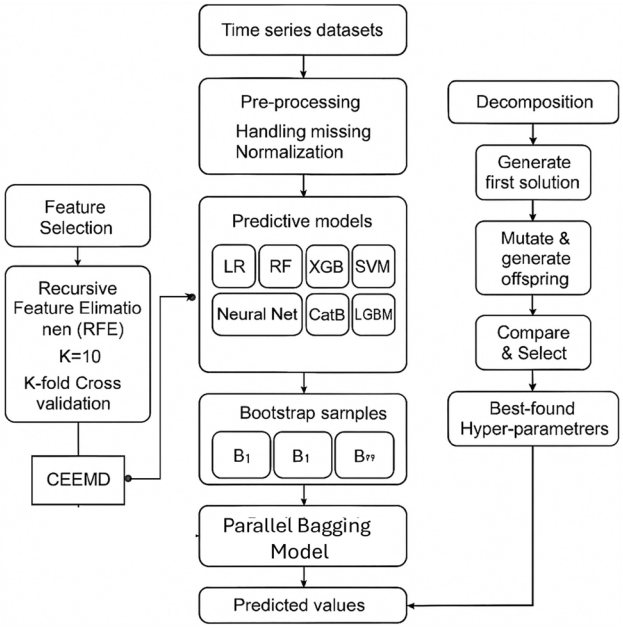


Fig. 3 Architecture of the EvoBagNet Framework for Stock Price Prediction. The pipeline integrates data preprocessing, feature selection via RFE, time series decomposition using CEEMD, bagging-based ensemble modeling with Extra Trees, CatBoost, and LightGBM, and hyperparameter optimization using a (1 + 1) evolutionary algorithm

4.2.2 Data normalization

In preprocessing, data normalization is the crucial step in statistical and ML modelling. It ensures that all features contribute equally, enhances the model's performance, and accelerates the convergence rate of optimization techniques. There are various approaches for data normalization, but in this study, min-max scaling is applied and is done by using Eq. (20)

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}} \tag{20}$$

X_{scaled} is the normalized feature value after applying min-max scaling, X is the actual feature value, X_{min} and X_{max} are the lowest and highest values of the feature in the dataset, respectively.

4.2.3 Feature selection

Machine Learning approaches perform better when features are carefully chosen. As observed from the literature survey, the stock market data is highly volatile and noisy, making the prediction very challenging; to improve the prediction accuracy, it is critical to comprehend the features and structure of the stock market data [61]. The following are the main reasons for the importance of feature selection. Firstly, feature reduction leads to fewer chances of model overfitting, and another reason is a better understanding of the features and their relationships with the target feature. Lastly, it reduces the computational time. There are different approaches to feature selection; in this study, the Recursive Feature Elimination (RFE) mechanism has been applied. This approach is widely used and is based on a greedy mechanism to choose the more relevant subset of features [62]. It is a wrapper method in which the score of each feature is calculated, and the feature with the lowest score is removed. This process is recursively repeated until the required feature set is obtained. The number of features to be chosen is given in advance to the algorithm. In this study, out of a total of 14 features, only nine relevant features were selected for further processing.

The overall architecture of the proposed EvoBagNet framework is illustrated in Fig. 3. The pipeline begins with raw time series data sourced from the National Stock Exchange (NSE), followed by preprocessing steps such as missing value imputation and normalization. Feature selection is performed using Recursive Feature Elimination (RFE) with 10-fold cross-validation to retain the most relevant input features. Subsequently, the

Table 4 Performance of SVR on top IT sector stocks

Data partitioning metric	70:30				80:20				90:10			
	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE
Infosys	1.42	0.93	27.21	20.66	1.32	0.90	26.12	18.39	1.07	0.83	21.41	15.81
TCS	1.59	0.90	52.08	38.33	1.19	0.91	47.65	33.18	1.03	0.83	56.02	36.63
Wipro	1.37	0.85	7.51	5.54	1.47	0.87	8.02	5.94	1.32	0.86	8.01	5.58
HCL	1.25	0.95	21.41	15.22	1.22	0.92	19.53	13.89	1.51	0.92	26.90	19.55
Mahindra	1.76	0.91	24.37	19.25	1.44	0.93	22.32	16.65	1.50	0.87	23.24	18.24
Mindtree	2.34	0.88	151.49	111.33	1.68	0.93	135.28	85.90	1.94	0.87	167.83	105.29
Coforge	2.15	0.90	202	120.82	2.02	0.92	192.02	109.52	2.52	0.87	219.01	137.94
Mphasis	2.20	0.92	60.02	46.30	1.70	0.96	49.73	36.39	1.67	0.92	53.85	38.81
L&T Tech	2.48	0.93	125.37	95.97	2.14	0.94	115.87	85.42	2.55	0.84	143.43	117.74

preprocessed data is decomposed using the Complete Ensemble Empirical Mode Decomposition (CEEMD), producing Intrinsic Mode Functions (IMFs) that capture various temporal dynamics of the stock signals.

4.2.4 Data partitioning:

After preprocessing, various commonly used ratios were examined for data partitioning and for training and testing the machine learning models. These divisions help in the efficient assessment and validation of the model. In the current study, three data partitioning ratios –70:30, 80:20, and 90:10 –were applied, and the results achieved with all these are discussed in the results section.

After completing the preprocessing, the next crucial step is implementing ML models, which involves selecting appropriate models, training and tuning them, and assessing their performance to ensure they provide reliable results.

5 Experimental evaluation

This section presents the performance of ML approaches to understanding the effectiveness of the proposed study. It evaluates the efficacy of various models for predicting the stock market using different data partitioning ratios. The change in the train test split ratio also impacts the reliability of the ML model [63]. The accuracy and stability of the model are enhanced by expanding the size of the training data. However, the drift is in a different direction when the latter has grown from 80 to 90%. Thus, the splitting ratio has a significant impact on the predictive power of the ML model.

This study applied nine ML techniques: SVR, RF, XGB, LR, DT, LGBM, GBR, CATb, and ANN to predict closing prices based on different splitting ratios of input data. The dataset was split into 30/70 and 90/10 training and testing splitting ratios, in addition to the most common ratio, the 80/20 ratio, and results achieved with all three splitting ratios were analyzed.

All approaches were implemented using Python and Keras, an open-source deep-learning library built on TensorFlow. The experiments were conducted in a computational environment featuring an Intel i7-8665U processor running at 2.11 GHz, 16 GB of RAM, and the Windows 10 Pro operating system. This setup ensured sufficient computational power and compatibility for training and evaluating the models.

In the first experiment, Support Vector Regression (SVR) was deployed on all nine datasets, with hyper-parameter tuning performed using grid search. The experiments were conducted using various train-test split ratios to analyze the model's performance under different data distributions. The results are presented in Table 4. As shown, the highest prediction accuracy was achieved for the HCL dataset, with an accuracy of 95% and a Mean Absolute Error (MAE) of 15.22. Moreover, the results indicate that SVR is highly sensitive to the percentage of training data. Specifically, when the training data exceeds 70%, the model tends to overfit, leading to a reduction in test accuracy. This highlights the importance of selecting an optimal train-test split to balance training sufficiency and generalization capability for robust predictions.

Next, the performance of Random Forest (RF) was evaluated using various train-test split ratios across nine stock datasets. At first glance, RF demonstrated better average performance compared to SVR for all datasets, primarily due to its ability to effectively capture complex and nonlinear relationships in the data. With the exception of Wipro and HCL, which achieved accuracies of 89% and 86%, respectively, RF predicted the next day's stock price with a high accuracy exceeding 90% for the remaining datasets. Additionally, it was observed that RF is less sensitive to the choice of train-test split ratios compared to SVM, making it more robust and reliable for stock price prediction tasks (Table 5).

XGBoost is a widely used method for stock price forecasting, and its performance was evaluated across nine case studies, as shown in Table 6. The highest accuracy was achieved for the Coforge dataset at 94%, while the lowest accuracy was observed for the HCL dataset at 83%. This variation in accuracy can be attributed to

Table 5 Performance of random forest on top IT sector stocks

Data partitioning metric	70:30				80:20				90:10			
	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE
Infosys	1.21	0.94	23.51	17.52	1.05	0.94	20.80	16.36	1.05	0.85	20.75	17.02
TCS	1.01	0.93	44.79	33.68	0.96	0.95	36.64	31.11	1.03	0.88	48.94	36.17
Wipro	1.20	0.89	6.29	4.76	0.97	0.94	4.90	4.31	1.06	0.91	6.45	4.44
HCL	1.58	0.86	38.58	19.94	1.32	0.92	35.22	18.64	2.04	0.79	51.77	27.59
Mahindra	1.41	0.93	21.75	16.25	1.41	0.93	21.10	15.72	1.59	0.86	23.11	18.99
Mindtree	1.62	0.94	102.65	78.14	1.33	0.96	86.31	66.81	1.34	0.93	89.11	71.61
Coforge	1.85	0.96	121.51	87.31	1.80	0.97	113.78	83.53	1.90	0.89	144.14	101.44
Mphasis	2.09	0.93	56.35	44.58	1.73	0.96	48.33	37.29	1.79	0.92	53.90	42.13
L&T Tech	1.66	0.96	86.20	64.17	1.17	0.97	69.32	51.76	1.45	0.94	84.36	65.12

differences in stock price volatility among companies. Stocks with more stable price movements tend to yield higher predictive accuracy, whereas higher volatility introduces greater uncertainty, making accurate forecasting more challenging.

Given its advantages in handling high-dimensional data, irrelevant or redundant features, and multi-collinearity, Lasso regression was employed in this study. Its ability to balance model simplicity with predictive performance makes it a robust tool for regression tasks. The results of Lasso regression for all case studies are summarized in Table 7. On average, Lasso regression delivered strong performance with an acceptable level of accuracy and MAE across most datasets. Notably, the highest accuracy was achieved for the HCL and LTI Mindtree datasets, both at 96%.

Another well-known machine learning model for stock price prediction is neural networks, renowned for their exceptional ability to extract nonlinear and complex patterns from data. This capability allows them to outperform traditional models, such as linear regression or decision trees, in capturing intricate relationships. Table 8 presents the prediction results of the NN model across nine case studies. Neural networks demonstrated effective performance for most datasets, achieving an accuracy exceeding 90%. Notably, the highest accuracy of 95% was observed for both the HCL and L&T Tech datasets.

5.1 Hyper-parameter tuning

Hyper-parameter tuning is a critical stage in the development of an ML model because hyper-parameters regulate the model's behavior during the training process. Selecting an optimal configuration that minimizes the error rate and maximizes accuracy can significantly enhance the model's performance on unseen data.

This study implements hyper-parameter tuning to enhance model performance and also proves to be a more reliable and precise prediction of the next day's closing price, indicating the effectiveness of our method in a volatile stock market environment. The hyperparameters, their corresponding value ranges, and the selected optimal values are presented in Tables 9, 10, and 11. In SVR, the trade-off between maximizing the margin and minimizing the training error is controlled by the regularization parameter *C*. If the value of C is small, that allows a wider margin, but it may lead to huge training errors. *epsilon* is the tolerance margin around the predicted value. It is the point at which training mistakes do not result in a penalty. It regulates the diameter of the tube, beyond which faults are deemed negligible.

In addition, *gamma* is used to characterize the influence of a single training sample, with high values indicating near and low values indicating far. It establishes how adaptable the decision limit is. In Random Forest, three hyper-parameters have been optimized: *n_estimators* controls the number of decision trees that will be running in the forest, *max_depth* controls the maximum depth of each decision tree, and *min_samples_split* represents the minimum number of samples required to split an internal node. In the case of the XGBoost approach, three hyperparameters are optimized: *n_estimators* and *max_depth* are already defined, and the third is *learning_rate*. The learning rate determines the step size at every boosting iteration. By taking fewer steps, lower learning rate values strengthen the model; however, it may require more boosting rounds for it to converge. In Lasso Regression, only one hyper-parameter is optimized; that is, α, alpha regulates the strength of the penalty term in the lasso objective function. It keeps the model simple and fits the training set well. Higher alpha values indicate more regularization, resulting in fewer coefficients approaching zero.

Hyper-parameter tuning is highly beneficial for ANN because of its numerous parameters. Six hyper-parameters are optimized, including epochs, learning rate (LR), hidden layers, activation function, dropout, and shape. The hyper-parameter *epoch* represents how often the training data is passed to the neural network, and in the proposed study, three values of epochs are supplied. An important hyper-parameter *lr* is used to control the model's step size during optimisation. Choosing the appropriate value of lr is crucial because a higher value may overshoot the optimal result, while using a lower value may result in a slower convergence. Another hyper-parameter is *hidden layers*, which represents the number of layers between the input and output layers. The activation function *ReLU* has been used because it is computationally efficient and involves simple thresholding.

Table 6 Performance of XGBoost on top IT sector stocks

Data partitioning metric	70:30				80:20				90:10			
	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE
Infosys	1.37	0.92	24.36	18.16	1.35	0.93	24.85	18.37	1.50	0.85	21.15	15.61
TCS	1.02	0.93	48.40	33.84	1.27	0.92	50.90	38.37	0.99	0.90	43.37	31.76
Wipro	1.59	0.85	7.55	5.81	1.35	0.91	7.06	5.30	1.44	0.88	7.44	6.00
HCL	1.83	0.83	43.47	23.16	1.73	0.91	37.88	20.65	2.38	0.75	59.96	32.20
Mahindra	1.46	0.92	22.54	16.76	1.40	0.93	20.82	15.73	2.20	0.64	34.18	27.61
Mindtree	2.10	0.91	131.84	101.52	1.39	0.96	90.48	69.45	1.53	0.91	103.49	81.37
Coforge	2.02	0.94	150.32	100.71	1.83	0.97	123.37	83.43	2.39	0.88	200.62	134.11
Mphasis	5.93	0.66	141.54	118.43	1.72	0.96	47.76	36.68	2.09	0.90	62.07	48.60
L&T Tech	2.10	0.89	131.84	101.52	1.39	0.95	90.48	69.45	1.51	0.91	103.49	81.37

https://doi.org/10.1007/s00521-025-11789-z

Table 7 Performance of lasso regression on top IT sector stocks

Data partitioning metric	70:30				80:20				90:10			
	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE
Infosys	1.08	0.95	21.75	15.60	1.07	0.93	20.05	14.87	0.95	0.87	18.78	13.89
TCS	0.89	0.94	40.49	29.78	0.83	0.94	38.59	28.34	0.83	0.90	42.36	29.29
Wipro	1.21	0.89	6.29	4.56	1.69	0.88	6.63	4.84	1.26	0.92	6.48	4.33
HCL	1.04	0.96	15.993	11.61	0.99	0.97	16.37	11.84	1.02	0.96	18.92	13.02
Mahindra	1.28	0.94	19.30	14.16	1.18	0.95	18.61	13.46	1.44	0.87	18.64	13.78
Mindtree	1.42	0.96	91.03	68.51	1.23	0.96	82.61	61.52	1.82	0.94	79.20	57.93
Coforge	1.52	0.95	99.37	76.75	1.42	0.96	98.92	69.65	1.59	0.94	107.88	72.98
Mphasis	1.47	0.96	42.43	31.57	1.43	0.97	42.62	30.85	1.55	0.93	48.84	36.32
L&T Tech	1.43	0.94	74.42	53.42	1.23	0.96	68.57	49.75	1.32	0.94	0.69	50.92

Table 8 Performance of ANN on top IT sector stocks dataset

Data partitioning metric	70:30				80:20				90:10			
	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE	MAPE	R^2	RMSE	MAE
Infosys	0.014	0.93	26.79	20.12	0.013	0.96	22.63	18.39	0.04	0.91	26.32	21.14
TCS	0.0099	0.92	25.79	33.68	0.0089	0.94	32.33	29.58	0.0094	0.93	46.87	31.58
Wipro	0.016	0.89	6.57	4.68	0.014	0.94	4.85	4.11	0.019	0.92	6.60	4.68
HCL	0.010	0.95	19.74	13.84	0.10	0.97	18.04	12.93	0.011	0.91	22.83	15.43
Mahindra	0.012	0.86	19.39	14.07	0.011	0.93	17.21	13.10	0.011	0.87	17.94	13.30
Mindtree	0.013	0.92	97.56	73.95	0.013	0.95	91.97	67.77	0.012	0.93	92.18	66.84
Coforge	0.016	0.94	115.38	82.38	0.013	0.96	109.01	74.59	0.014	0.93	123.56	81.02
Mphasis	0.019	0.92	56.96	42.88	0.016	0.95	51.84	38.31	0.016	0.91	54.22	39.11
L&T Tech	0.013	0.95	80.17	60.07	0.013	0.97	76.89	55.78	0.014	0.93	89.69	66.92

 https://doi.org/10.1007/s00521-025-11789-z

Table 9 Hyper-parameters and their selected values of SVR and random forest

SVR			Random forest		
Hyper-parameter	Values	Selected value	Hyper-parameter	Values	Selected value
C	[0.1, 1, 10, 100]	100	n_estimators	[100, 200, 300, 400]	100
Gamma	[0.1, 0.01, 0.001]	0.01	Max_depth	[3, 4, 5, 6, 7]	5
Epsilon	[0.1, 0.01, 0.001]	0.1	Min_samples_split	[2, 3, 4,5, 10]	3

Table 10 Hyper-parameters and their selected values of XGBoost and lasso regression

XGBoost			Lasso regression		
Hyperparameter	Values	Selected value	Hyper-parameter	Values	Selected value
n_estimators	[200, 300, 400]	400	Alpha	[0.001, 0.01, 0.1, 1, 10]	1
Max_depth	[3, 5, 7]	7			
Learning_rate	[0.02, 0.03, 0.05, 0.1, 0.2]	0.1			

Table 11 Hyper-parameters and their best-selected values of artificial neural network

Hyper-parameter	Values	Selected value	Hyper-parameter	Values	Selected value
Epochs	[300, 400, 500]	500	Dropout	(0–0.5)3 levels	0.0
lr	(0.01–0.05) 3 levels	0.036	Activation function	[Relu]	Relu
Hidden layers	[1, 2, 3]	2	Shape	[Brick, Funnel]	Funnel

Sometimes the model leads to overfitting; *dropout* is used to control the overfitting. The range of value is passed to the model for selecting the optimal one; the last hyper-parameter is the *shape* of the neural network, which represents the structural design of neurons and can be either brick or funnel. This hyper-parameter also impacts the learning capability of the network. In ANN, a validation set has been used to adjust the hyper-parameters and to avoid overfitting. This also helps ensure the model generalizes effectively to new data. In this, the 'Scan' function of the Talos library has been used to traverse both training and validation sets for the adjustment of hyper-parameters. This function analyzes all the permutations and combinations of hyper-parameter values that are supplied, as shown in Table 10. Another object in the same library is 'Analyze', which is used to carry out all the outcomes of the tuning activity. To obtain the optimal configuration of the model, *Talos.best_model(metric: low MAPE)* is used, and this model is the final optimized model that is finally applied to the test set.

As the complexity and volume of financial data continue to increase, practical visualization approaches are becoming increasingly necessary for interpreting and understanding predictive models in the equity market domain.

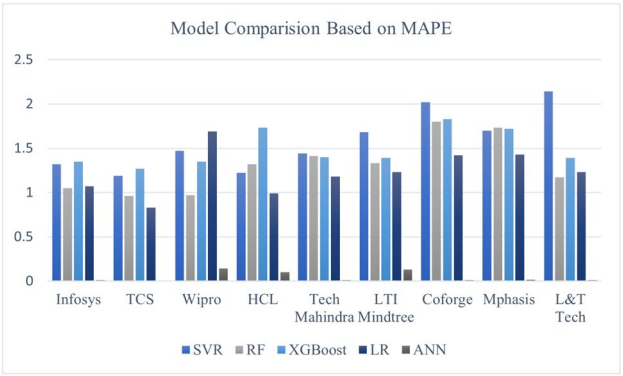
This research investigates the utility of interactive line plots as a visualization aid for displaying the real price and the forecasted price. While using the interactive nature of line plots, users may examine model performance by zooming in/Out, hovering over data points, and pinpointing possible areas for improvement by panning between periods. In the proposed study, only the best results achieved are shown using the interactive line plot. The proposed training set is used for training the model, and the models achieved by training are used to predict the test set data and the real value is compared with the predicted value as shown in Fig. 4a–o. In this Figures (a), (b), and (c) represent the actual stock price and predicted stock price using SVR applied on Infosys, TCS, and Wipro, Figures (d), (e), and (f) show the prediction of Random forest applied on same datasets. Similarly, the remaining Figures show the prediction of XGB, Lasso regression, and optimized ANN. Based on the actual values and predicted values obtained using each method, the error calculation for each technique is performed, and a final comparison of the results is made.

Fig. 4 comparative line chart between actual and predicted price using different ML approaches

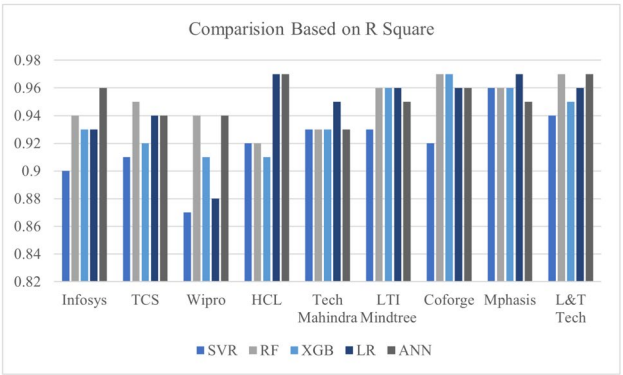
5.2 Comparative analysis

This section compares the proposed approaches based on the evaluation metrics. As already known, when the MAPE value is close to zero, and R^2 is close to 1, the model performs well. One example of individual ML models' performance for three datasets can be seen in Fig. 5, and it is interpreted from the chart that ANN with deep

 https://doi.org/10.1007/s00521-025-11789-z



(a)



(b)

Fig. 5 Comparison of proposed ML approaches based on MAPE and R square on Infosys, TCS, and Wipro

hyper-parameter tuning showed better results as compared to the others. The other two evaluation metrics, RMSE and MAE, are scale-dependent; that is, if the datasets are of different scales, then using RMSE and MAE in this manner may not provide a fair model comparison. In our study, the three datasets have different scales, which is the main reason for comparing the model results (based on RMSE and MAE) differently, as shown in Fig. 6.

To provide a more accurate comparison among the six proposed bagging models, we present Table 12. The table extensively evaluates the performance of these models on the Coforge time series data using six distinct metrics. As illustrated in Table 12, all proposed models achieved high prediction accuracy, exceeding 97%. Notably, the best-performing model was Bag-DT, which attained an accuracy of 98.5%, demonstrating its superior capability in forecasting stock prices.

For the L&T Tech dataset, all proposed ensemble models demonstrated exceptional performance, achieving a prediction accuracy above 98% (R^2), as shown in Table 13. Among these, Bag-LGBM and Bag-RF outperformed the other models in terms of Mean Absolute Error (MAE) and Root Mean Square Error (RMSE). A key observation from these results is the robustness of the proposed models, as evidenced by their consistent performance across K-fold cross-validation. Furthermore, the low variability in predictions, indicated by a small standard deviation (STD), underscores the model's ability to reliably capture learned patterns without significant deviations, enhancing its dependability for accurate forecasting.

Table 14 presents the price prediction results for the Mphasis dataset using the six proposed bagging models. Among these, Bag-GBR (Bagging with Gradient Boosting Regressor) and Bag-RF (Bagging with Random Forest) demonstrated superior performance, achieving significantly higher accuracy compared to the other predictors. Notably, these models improved prediction accuracy by approximately 3% over their original counterparts, such as Random Forest (RF) and Extreme Gradient Boosting (XGB), highlighting the effectiveness of the bagging approach in enhancing model robustness and predictive power. The best-performing prediction results are highlighted in bold font.

Table 15 highlights the impressive performance of bagging models in predicting stock prices for the Wipro dataset, achieving an R^2 score exceeding 96%. Among these, the Bag-GBR model demonstrated the highest accuracy at 97.4%. This exceptional performance can be attributed to Gradient Boosting's iterative ensemble approach, where each subsequent model focuses on minimizing the errors of its predecessors. This sequential learning process enables Bag-GBR to effectively capture intricate, non-linear patterns in the data, which is crucial for modelling the inherently volatile and multifaceted nature of stock price movements. The best-performing prediction results are highlighted in bold font.

Table 16 presents the performance of six bagging ensemble models trained and tested on the TCS dataset, evaluated across six key regression metrics. Among the models, Bag-RF achieved the best overall performance, attaining the highest mean R^2R^2 value of 0.973, the lowest RMSE of 26.845, and the lowest SMAPE of 0.599, indicating strong predictive accuracy and stability. Bag-GBR followed closely, with a mean R^2 of 0.970 and RMSE of 28.136. On the other hand, Bag-CatB and Bag-LGBM exhibited relatively weaker performance, with mean R^2 scores of 0.962 and 0.964, respectively, and higher error metrics. Bag-XGB and Bag-DT produced similar outcomes, with mean RMSE values around 29.2 and R^2 near 0.968–0.969. Across all models, the standard deviations of the metrics were low, demonstrating consistency in performance. These results highlight Bag-RF as the most effective and reliable ensemble model for the TCS dataset. The best-performing prediction results are highlighted in bold font.

Table 17 compares the performance of six bagging ensemble models evaluated on the Infosys dataset. Among them, Bag-GBR achieved the best overall results, with the highest mean R^2 of 0.973, the lowest RMSE (17.861), and the lowest MSLE (1.87E−04), indicating strong predictive accuracy and generalization. Bag-XGB also

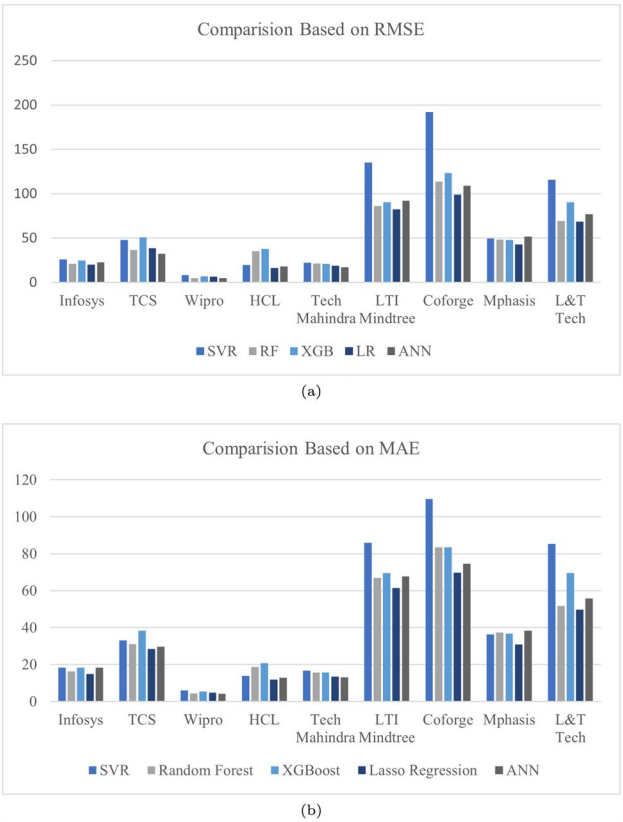


Fig. 6 Comparison of proposed ML approaches based on RMSE and MAE on Infosys, TCS, and Wipro

demonstrated excellent performance with a slightly lower R^2 of 0.968 and the lowest SMAPE value of 0.840,

Table 12 The performance of bagging ensemble models trained and tested by coforge dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	100.938	64.378	0.984	3.54E−04	0.980	1.294	Mean	205.939	145.449	0.975	1.49E−03	0.944	2.886
Min	77.708	46.616	0.979	2.03E−04	0.974	0.936	Min	149.168	95.452	0.969	7.37E−04	0.926	1.883
Max	116.047	77.434	0.989	4.81E−04	0.987	1.568	Max	256.288	189.583	0.978	2.32E−03	0.963	3.777
STD	9.846	7.788	0.003	7.03E−05	0.003	0.159	STD	30.104	26.197	0.003	4.44E−04	0.011	0.525
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	106.100	68.973	0.984	4.01E−04	0.979	1.393	Mean	89.655	59.760	0.985	3.01E−04	0.983	1.224
Min	87.797	53.927	0.979	2.69E−04	0.971	1.095	Min	65.135	40.200	0.973	1.41E−04	0.968	0.808
Max	138.063	94.271	0.989	6.51E−04	0.984	1.880	Max	120.208	79.706	0.992	5.25E−04	0.991	1.637
STD	17.845	14.700	0.003	1.37E−04	0.004	0.289	STD	15.974	12.344	0.006	1.16E−04	0.007	0.262
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	101.850	65.997	0.984	3.79E−04	0.981	1.336	Mean	127.512	68.004	0.969	5.17E−04	0.966	1.345
Min	77.221	43.579	0.979	1.92E−04	0.971	0.871	Min	118.512	58.441	0.960	4.27E−04	0.957	1.143
Max	140.132	98.873	0.990	7.12E−04	0.988	2.008	Max	142.817	83.664	0.973	6.84E−04	0.970	1.676
STD	22.235	18.787	0.003	1.79E−04	0.005	0.380	STD	7.058	7.500	0.004	7.47E−05	0.004	0.159

https://doi.org/10.1007/s00521-025-11789-z

Table 13 Comparison of bagging ensemble models performance trained and tested by L&T Tech dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	65.882	50.526	0.984	2.62E−04	0.983	1.235	Mean	85.508	65.929	0.987	3.95E−04	0.982	1.569
Min	49.867	37.859	0.978	1.46E−04	0.978	0.922	Min	71.427	53.682	0.982	2.73E−04	0.974	1.279
Max	83.880	64.395	0.990	4.11E−04	0.990	1.576	Max	104.814	84.718	0.991	5.85E−04	0.987	2.017
STD	9.397	7.236	0.004	7.42E−05	0.004	0.178	STD	10.803	9.610	0.003	9.81E−05	0.004	0.226
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	58.092	43.804	0.987	2.02E−04	0.987	1.072	Mean	65.882	50.526	0.984	2.62E−04	0.983	1.235
Min	50.957	38.588	0.984	1.53E−04	0.984	0.943	Min	49.867	37.859	0.978	1.46E−04	0.978	0.922
Max	65.611	49.534	0.990	2.57E−04	0.990	1.215	Max	83.880	64.395	0.990	4.11E−04	0.990	1.576
STD	5.847	4.333	0.003	4.05E−05	0.002	0.106	STD	9.397	7.236	0.004	7.42E−05	0.004	0.178
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	55.612	42.034	0.988	1.88E−04	0.988	1.025	Mean	55.633	42.124	0.988	1.94E−04	0.988	1.035
Min	37.265	28.219	0.981	8.31E−05	0.981	0.691	Min	39.557	29.574	0.980	9.36E−05	0.980	0.724
Max	70.916	53.880	0.995	2.98E−04	0.995	1.317	Max	74.546	57.639	0.994	3.64E−04	0.994	1.452
STD	10.168	7.839	0.004	6.46E−05	0.004	0.191	STD	11.016	8.582	0.004	8.22E−05	0.005	0.219

https://doi.org/10.1007/s00521-025-11789-z

Table 14 Comparison of Bagging ensemble models performance trained and tested by Mphasis dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	31.286	23.853	**0.987**	2.21E−04	0.986	1.121	Mean	52.870	40.924	0.976	7.30E−04	0.969	2.006
Min	25.489	19.256	0.979	1.39E−04	0.979	0.901	Min	44.602	34.277	0.969	5.14E−04	0.959	1.686
Max	42.831	34.441	0.991	4.64E−04	0.991	1.686	Max	71.921	55.602	0.982	1.36E−03	0.977	2.745
STD	5.764	4.814	0.004	1.01E−04	0.004	0.243	STD	8.482	6.534	0.004	2.64E−04	0.007	0.329
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	35.781	27.841	0.984	2.91E−04	0.983	1.310	Mean	37.618	29.201	0.980	3.16E−04	0.979	1.365
Min	22.882	17.598	0.976	1.16E−04	0.976	0.832	Min	27.452	20.800	0.973	1.60E−04	0.965	0.972
Max	47.599	39.588	0.993	5.41E−04	0.993	1.899	Max	53.132	43.261	0.989	6.69E−04	0.989	2.075
STD	6.631	5.784	0.005	1.13E−04	0.005	0.280	STD	7.432	6.315	0.006	1.47E−04	0.007	0.311
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	30.698	23.629	**0.987**	2.10E−04	0.987	1.109	Mean	38.160	30.179	0.983	3.54E−04	0.982	1.431
Min	25.063	19.677	0.983	1.38E−04	0.981	0.923	Min	25.955	19.655	0.975	1.51E−04	0.971	0.928
Max	40.084	31.633	0.992	3.91E−04	0.992	1.534	Max	59.667	49.418	0.991	8.66E−04	0.990	2.385
STD	4.331	3.466	0.003	7.23E−05	0.003	0.176	STD	9.818	8.676	0.005	2.17E−04	0.006	0.429

Table 15 Comparison of Bagging ensemble models performance trained and tested by Wipro dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	3.992	3.041	**0.974**	9.93E−05	0.973	0.758	Mean	7.045	5.272	0.961	3.26E−04	0.924	1.334
Min	2.880	2.081	0.967	4.98E−05	0.961	0.519	Min	5.803	4.283	0.948	2.20E−04	0.900	1.085
Max	5.317	4.246	0.984	1.80E−04	0.984	1.071	Max	8.150	6.386	0.975	4.29E−04	0.948	1.617
STD	0.689	0.618	0.006	3.62E−05	0.007	0.156	STD	0.773	0.745	0.010	7.04E−05	0.014	0.188
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	4.607	3.543	0.971	1.41E−04	0.967	0.888	Mean	4.949	3.765	0.966	1.64E−04	0.961	0.941
Min	3.341	2.478	0.955	6.82E−05	0.950	0.621	Min	3.345	2.463	0.947	6.64E−05	0.942	0.613
Max	8.188	7.365	0.982	4.25E−04	0.980	1.845	Max	8.999	7.834	0.981	4.91E−04	0.979	1.937
STD	1.358	1.396	0.007	1.04E−04	0.008	0.350	STD	1.584	1.523	0.011	1.21E−04	0.012	0.374
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	4.222	3.171	0.972	1.18E−04	0.968	0.793	Mean	4.080	3.046	**0.974**	1.03E−04	0.969	0.762
Min	3.000	2.215	0.948	5.44E−05	0.944	0.554	Min	3.322	2.453	0.964	6.71E−05	0.951	0.614
Max	6.889	5.758	0.985	3.02E−04	0.984	1.450	Max	5.182	3.920	0.983	1.64E−04	0.980	0.983
STD	1.280	1.115	0.013	7.93E−05	0.016	0.282	STD	0.629	0.504	0.007	3.28E−05	0.009	0.127

https://doi.org/10.1007/s00521-025-11789-z

Table 16 Comparison of Bagging ensemble models' performance trained and tested by the TCS dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	28.136	21.077	0.970	6.91E−05	0.969	0.620	Mean	34.198	24.650	0.962	9.77E−05	0.958	0.721
Min	21.359	15.868	0.955	3.84E−05	0.955	0.466	Min	29.578	21.020	0.948	7.24E−05	0.939	0.615
Max	34.487	26.182	0.984	1.02E−04	0.983	0.770	Max	40.896	29.511	0.971	1.37E−04	0.968	0.862
STD	4.828	3.663	0.010	2.32E−05	0.010	0.108	STD	3.861	2.913	0.009	2.25E−05	0.010	0.085
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	29.162	22.062	0.969	7.42E−05	0.968	0.651	Mean	29.513	22.503	0.967	7.60E−05	0.967	0.663
Min	22.027	16.435	0.956	4.13E−05	0.954	0.484	Min	20.937	16.470	0.954	3.75E−05	0.954	0.485
Max	36.039	27.794	0.982	1.14E−04	0.982	0.823	Max	34.985	26.461	0.984	1.05E−04	0.984	0.779
STD	3.937	3.175	0.008	2.04E−05	0.008	0.095	STD	4.478	3.121	0.009	2.16E−05	0.009	0.092
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	26.845	20.337	**0.973**	6.38E−05	0.972	0.599	Mean	30.865	23.182	0.964	8.37E−05	0.963	0.683
Min	14.789	11.288	0.960	1.85E−05	0.960	0.332	Min	23.148	17.243	0.952	4.60E−05	0.952	0.509
Max	33.110	25.133	0.992	9.37E−05	0.992	0.741	Max	35.650	26.385	0.981	1.09E−04	0.980	0.778
STD	5.502	4.194	0.010	2.29E−05	0.010	0.124	STD	5.077	3.752	0.012	2.59E−05	0.012	0.111

Table 17 Comparison of Bagging ensemble models' performance trained and tested by the Infosys dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	17.861	13.844	**0.973**	1.87E−04	0.971	1.004	Mean	32.371	25.673	0.958	5.94E−04	0.928	1.891
Min	11.221	8.060	0.960	6.73E−05	0.955	0.583	Min	28.454	22.031	0.941	4.55E−04	0.910	1.624
Max	27.726	23.081	0.985	4.15E−04	0.984	1.677	Max	38.358	32.335	0.971	8.23E−04	0.937	2.371
STD	5.663	5.520	0.008	1.23E−04	0.009	0.403	STD	3.266	3.327	0.009	1.21E−04	0.007	0.241
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	16.510	11.565	0.968	1.50E−04	0.967	0.840	Mean	18.864	13.910	0.958	1.99E−04	0.957	1.006
Min	13.494	9.303	0.952	9.85E−05	0.951	0.676	Min	12.299	8.771	0.937	8.01E−05	0.936	0.633
Max	20.374	14.470	0.980	2.22E−04	0.978	1.050	Max	25.784	20.453	0.981	3.57E−04	0.980	1.486
STD	2.582	1.845	0.011	4.59E−05	0.010	0.133	STD	4.683	3.926	0.016	9.58E−05	0.016	0.287
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	20.450	15.724	0.964	2.46E−04	0.963	1.138	Mean	20.178	15.468	0.968	2.31E−04	0.965	1.122
Min	13.130	9.191	0.949	9.15E−05	0.949	0.665	Min	14.152	10.184	0.956	1.08E−04	0.955	0.741
Max	35.911	32.037	0.978	7.00E−04	0.978	2.327	Max	28.408	24.716	0.981	4.34E−04	0.979	1.793
STD	6.855	7.135	0.009	1.84E−04	0.008	0.519	STD	5.170	5.575	0.008	1.20E−04	0.008	0.406

https://doi.org/10.1007/s00521-025-11789-z

Table 18 Comparison of Bagging ensemble models' performance trained and tested by the Mindtree dataset

Bag-GBR							Bag-CatB						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	61.017	47.471	**0.983**	1.51E−04	0.983	0.945	Mean	72.462	55.105	0.981	2.03E−04	0.979	1.092
Min	47.741	36.693	0.977	8.92E−05	0.977	0.728	Min	54.385	40.646	0.974	1.11E−04	0.974	0.801
Max	71.063	56.562	0.990	2.07E−04	0.990	1.147	Max	82.562	63.265	0.990	2.61E−04	0.988	1.250
STD	9.310	7.539	0.005	4.59E−05	0.005	0.154	STD	9.055	7.378	0.004	4.92E−05	0.005	0.148
Bag-XGB							Bag-DT						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	66.714	50.436	0.981	1.79E−04	0.980	1.008	Mean	72.889	55.719	0.980	2.14E−04	0.978	1.116
Min	57.471	43.747	0.976	1.31E−04	0.975	0.873	Min	62.317	48.643	0.971	1.55E−04	0.971	0.974
Max	81.635	63.628	0.986	2.61E−04	0.985	1.270	Max	85.152	64.078	0.988	2.85E−04	0.987	1.284
STD	7.293	5.889	0.003	3.98E−05	0.004	0.118	STD	7.442	5.401	0.005	4.62E−05	0.005	0.112
Bag-RF							Bag-LGBM						
Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE	Metric	RMSE	MAE	R^2	MSLE	EVS	SMAPE
Mean	62.899	47.496	**0.983**	1.61E−04	0.983	0.950	Mean	67.270	51.878	0.982	1.85E−04	0.981	1.036
Min	49.201	36.917	0.978	9.66E−05	0.977	0.737	Min	51.382	39.435	0.973	1.07E−04	0.973	0.790
Max	81.348	61.045	0.989	2.56E−04	0.989	1.213	Max	92.001	72.967	0.989	3.39E−04	0.989	1.463
STD	8.793	6.586	0.003	4.47E−05	0.004	0.131	STD	11.424	9.425	0.005	6.61E−05	0.005	0.190

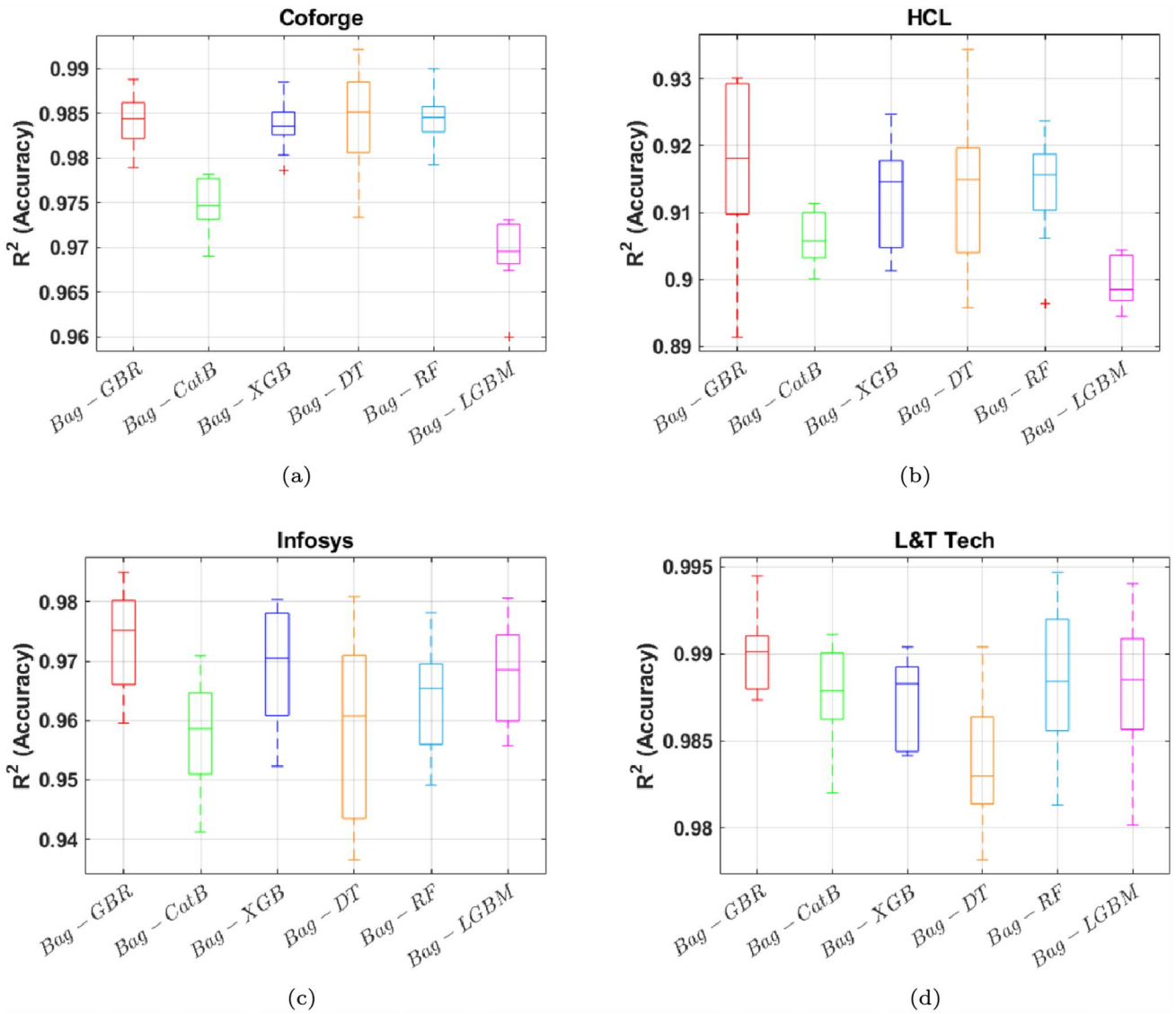


Fig. 7 Statistical results of Bagging ensemble models performance for four NSE datasets based on R^2

suggesting highly precise percentage-based error reduction. In contrast, Bag-CatB performed the weakest, with the highest RMSE (32.371) and SMAPE (1.891) and the lowest explained variance score (EVS = 0.928). Bag-LGBM and Bag-RF showed competitive results, both achieving similar R^2 values (0.968 and 0.964, respectively) and moderate error metrics. Although Bag-DT achieved a respectable R^2 of 0.958, it exhibited higher variability in performance, as reflected by its standard deviations. Overall, Bag-GBR and Bag-XGB stand out as the most robust and consistent models for this dataset. The best-performing prediction results are highlighted in bold font.

Table 18 presents the performance comparison of six bagging ensemble models on the Mindtree dataset. Bag-GBR and Bag-RF delivered the strongest performance, both achieving the highest mean R^2 of 0.983, with Bag-GBR having the lowest RMSE (61.017) and MSLE (1.51E−0), and Bag-RF slightly outperforming in SMAPE (0.950). Bag-XGB also performed competitively, with a mean R^2 of 0.981 and relatively low error metrics, while maintaining stable performance (the lowest standard deviation in R^2 was 0.003). Bag-LGBM exhibited slightly higher variability and error, with a mean RMSE of 67.270 and the highest standard deviation in RMSE (11.424), although it maintained a strong R-squared value of 0.982. Bag-CatB and Bag-DT had the weakest relative performance, both with the highest average RMSE values (72.462 and 72.889, respectively) and SMAPE above 1.09,

 https://doi.org/10.1007/s00521-025-11789-z

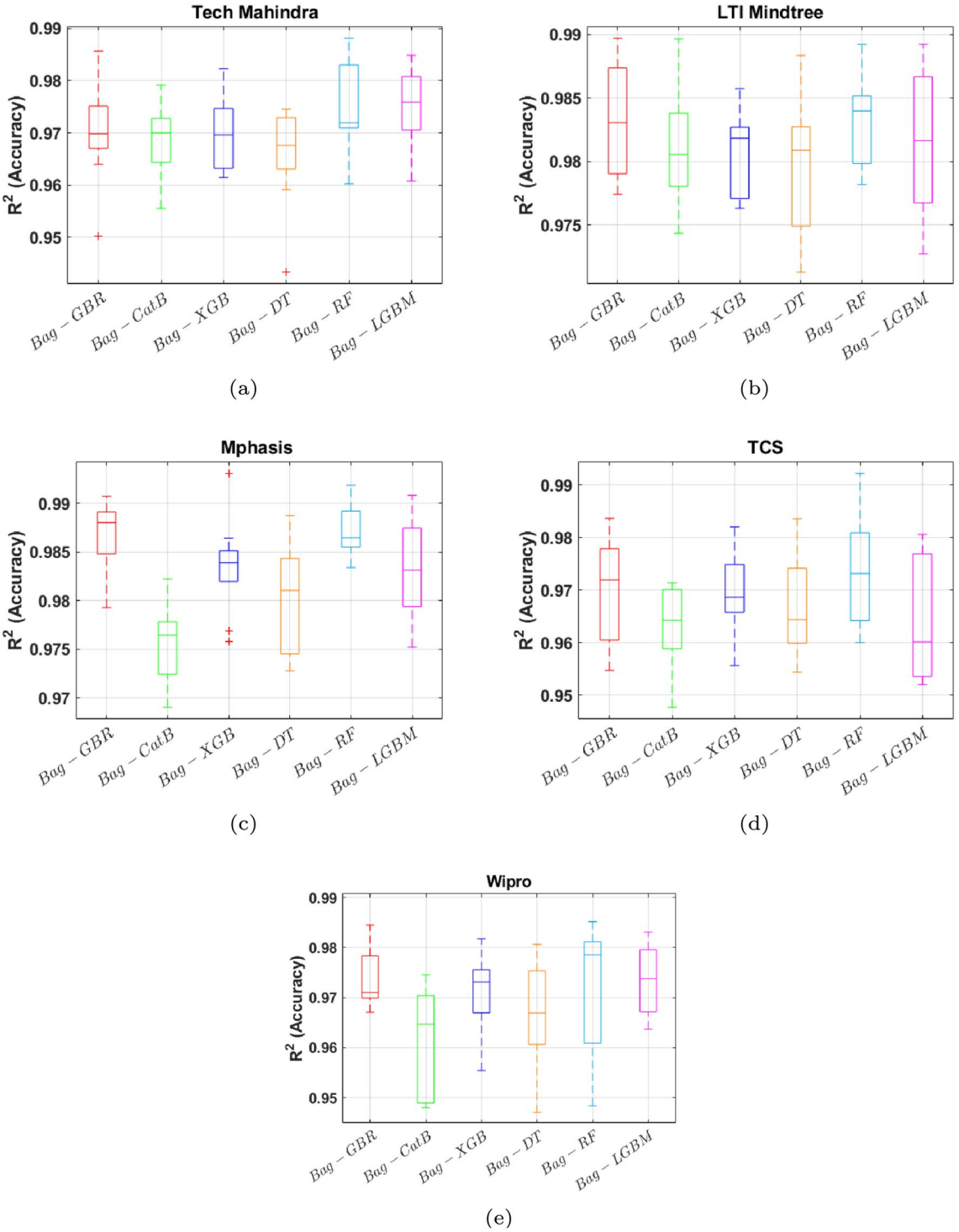


Fig. 8 Statistical results of Bagging ensemble models performance for five NSE datasets based on R^2

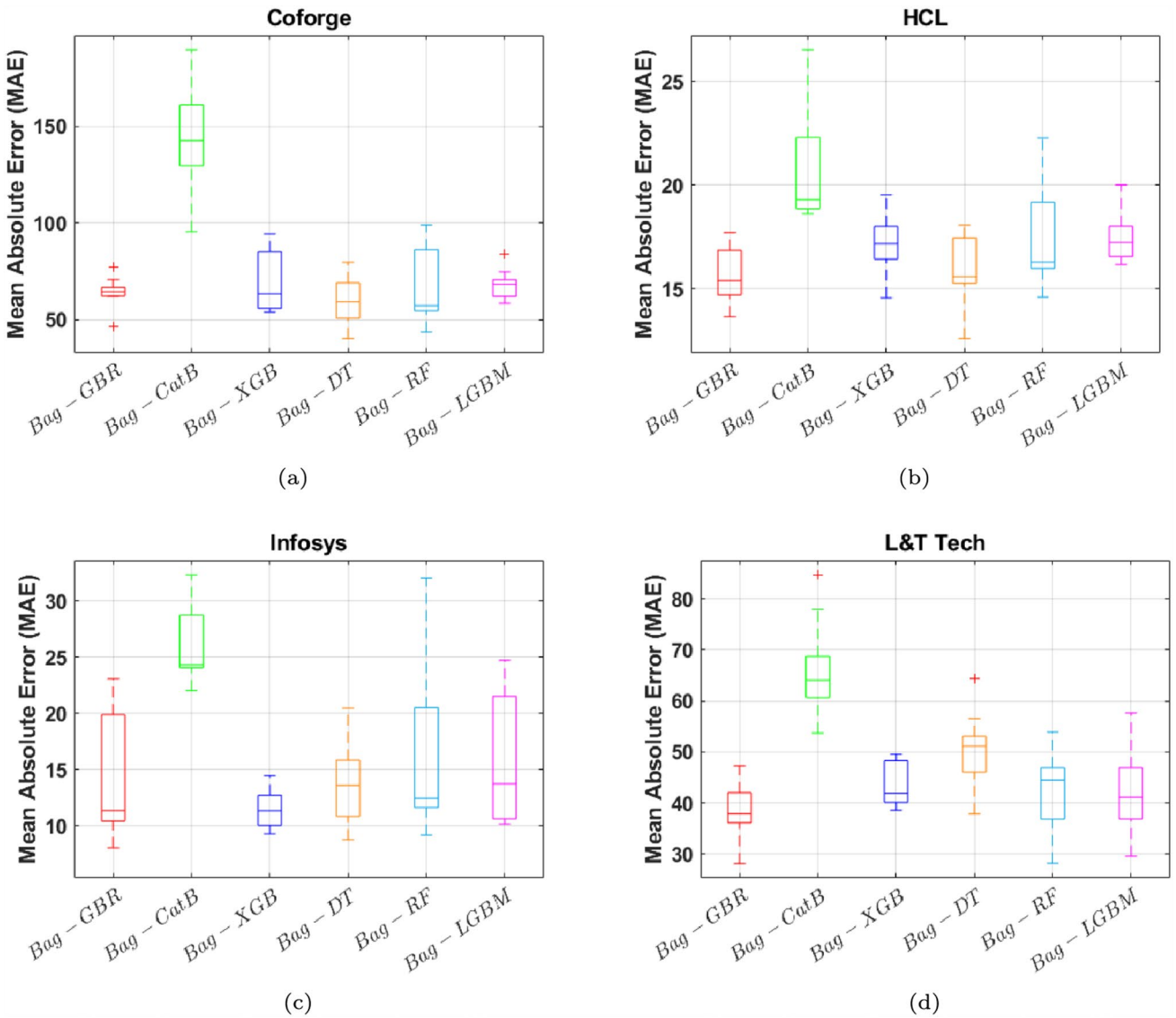


Fig. 9 Statistical results of Bagging ensemble models' performance for four NSE datasets based on MAE

although they still achieved respectable R^2 values around 0.980. Overall, Bag-GBR and Bag-RF emerge as the top-performing models for the Mindtree dataset in terms of both accuracy and consistency. The best-performing prediction results are highlighted in bold font.

Figures 7 and 8 present the statistical performance of the bagging models across nine datasets using the R^2 metric. Notably, the Bag-GRB model demonstrated superior performance compared to other bagging ensemble models in most case studies. It also exhibited lower variance in performance across multiple training and testing splits, indicating higher stability and robustness. Among the datasets, the HCL dataset posed the greatest challenge due to the high dynamics of its stock price. Despite this, the prediction accuracy of all bagging models remained significant and reliable across all nine case studies, showcasing their effectiveness in handling diverse datasets.

Figures 9 and 10 provide a comprehensive comparison of the six proposed bagging models across nine datasets based on the Mean Absolute Error (MAE). Among these, Bag-GBR consistently achieved the lowest learning error in most case studies, demonstrating its superior predictive accuracy. However, Bag-LGBM exhibited notable performance specifically for the Tech Mahindra and TCS datasets, outperforming other models in these

 https://doi.org/10.1007/s00521-025-11789-z

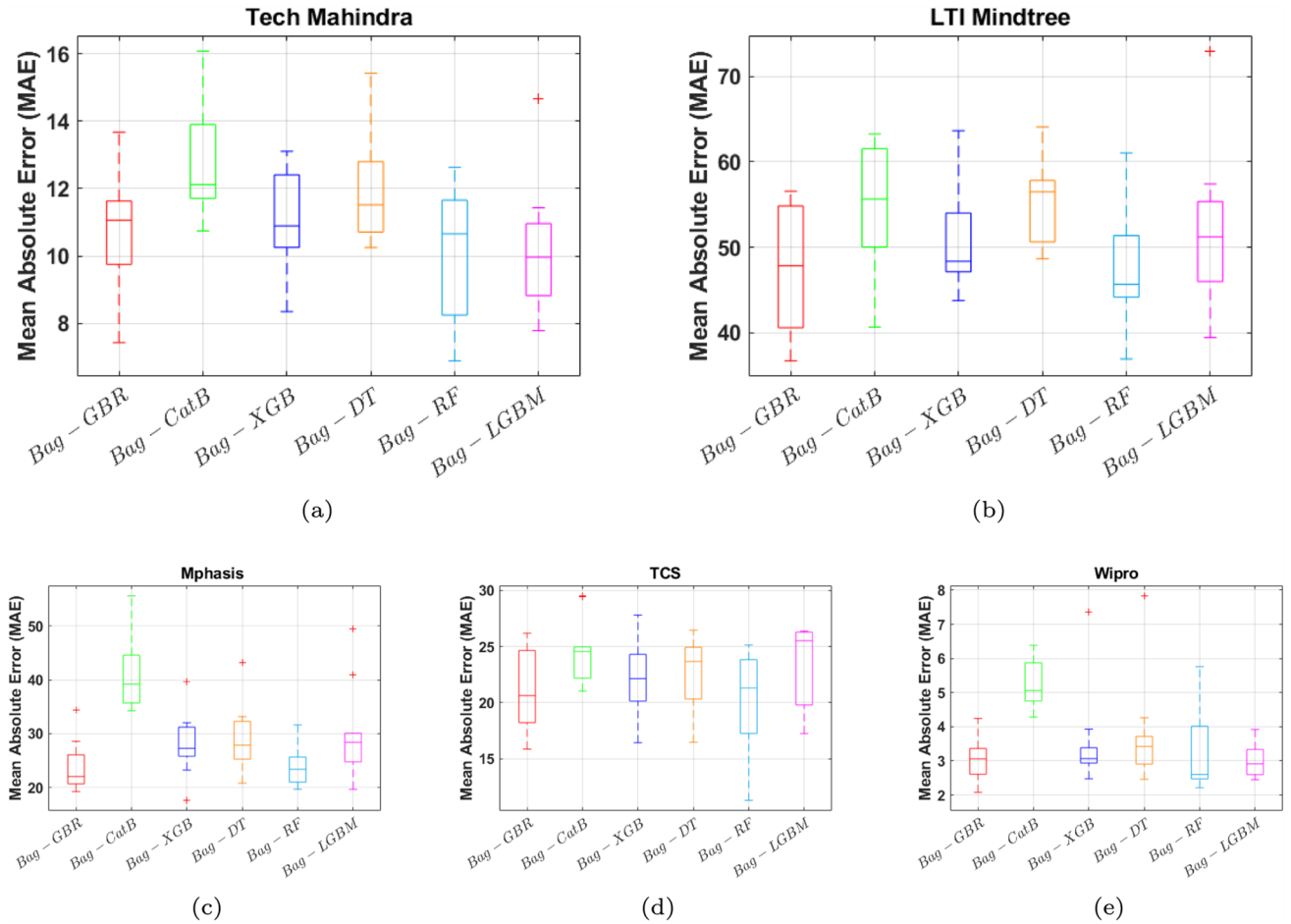
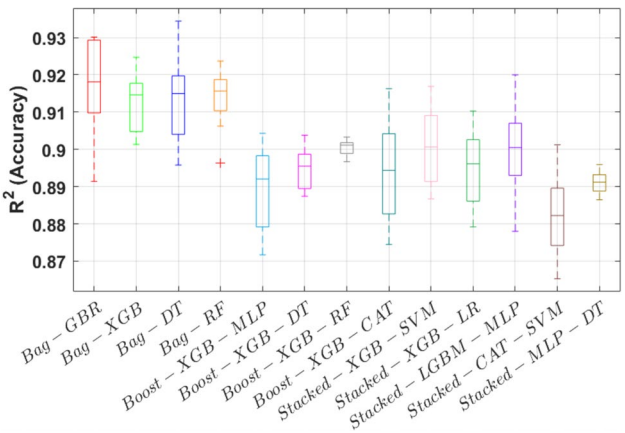


Fig. 10 Statistical results of Bagging ensemble models performance for five NSE datasets based on MAE

Fig. 11 Comparison of proposed EvoBagNet with other popular ensemble models based on R^2 for HCL



cases. These findings highlight the efficiency and reliability of the proposed models in accurately forecasting stock prices in advance.

Table 19 *P* values from statistical comparisons between EvoBagNet and each of the 12 ensemble models. Lower *p* values indicate statistically significant performance differences

Model	*p* value	Significant ($\alpha = 0.05$)?
Boost-XGB-RF	2.30×10^{-25}	Yes
Boost-XGB-DT	1.77×10^{-21}	Yes
Stacked-MLP-DT	1.00×10^{-23}	Yes
Stacked-XGB-SVM	5.82×10^{-19}	Yes
Stacked-CAT-SVM	9.81×10^{-19}	Yes
Stacked-XGB-LR	1.02×10^{-18}	Yes
Boost-XGB-MLP	1.48×10^{-18}	Yes
Stacked-LGBM-MLP	2.54×10^{-18}	Yes
Boost-XGB-CAT	8.95×10^{-18}	Yes
Bag-DT	3.53×10^{-3}	Yes
Bag-RF	4.90×10^{-3}	Yes
Bag-XGB	5.03×10^{-3}	Yes

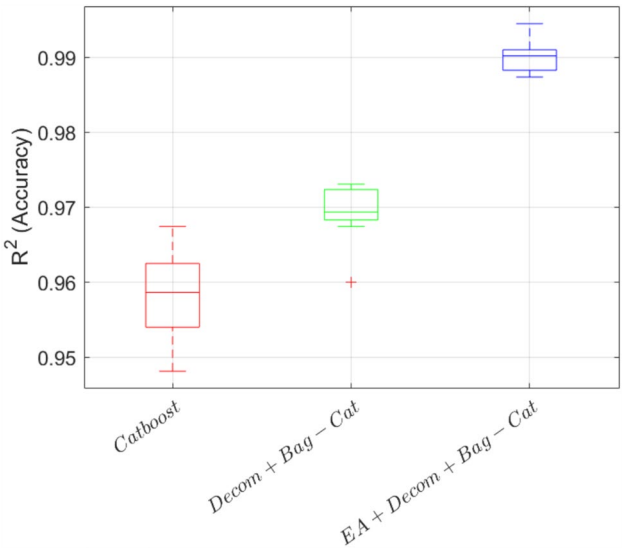


Fig. 12 Comparison of the proposed EvoBagNet's performance without decomposition and hyperparameter optimization, based on R^2 values, on the L&T-Tech dataset

The boxplot in Fig. 11 presents a comparative analysis of prediction accuracy, measured by R^2, across multiple ensemble learning models applied to IT sector stock price forecasting. Three groups of ensemble techniques are evaluated: bagging, boosting, and stacking. The first four boxplots represent bagging models (e.g., Bag–GBR, Bag–XGB, Bag–DT, Bag–RF), all of which consistently demonstrate higher median R^2 scores and tighter interquartile ranges, indicating both superior accuracy and lower variance in prediction performance. Among them, Bag–GBR achieves the highest median R^2, closely followed by Bag–XGB and Bag–DT.

In contrast, the boosting models (Boost–XGB–MLP, Boost–XGB–DT, Boost–XGB–RF, Boost–XGB–CAT) exhibit more dispersed distributions and generally lower median R^2 values. This suggests less stable performance and greater sensitivity to data variation. Similarly, the five stacked models show broader interquartile ranges and a wider spread of outliers, particularly for Stacked–SVM–DT and Stacked–LR–MLP, indicating variability and reduced robustness across datasets. Overall, the visual evidence strongly supports the efficacy of bagging-based models in this application. They outperform both boosting and stacking strategies in terms of both central tendency and consistency of accuracy, validating the methodological decision to adopt bagging ensembles as the core of the EvoBagNet framework.

The *p* value results presented in Table 19 indicate that the proposed model, EvoBagNet, demonstrates statistically significant performance differences when compared to the 12 ensemble models. Specifically, the majority of *p* values are below the conventional significance threshold (e.g., $p < 0.05$), suggesting that EvoBagNet

consistently outperforms the baseline ensemble methods in a meaningful way. These results support the robustness and effectiveness of the proposed approach, confirming that the observed performance improvements are unlikely to be due to random chance.

The boxplot in Fig. 12 compares the R^2 (accuracy) values for three different modeling approaches: CatBoost, Decomposition + Bagging + CatBoost, and the proposed EvoBagNet model, which integrates EA with Decomposition and Bagging strategies before applying CatBoost. The baseline CatBoost model shows relatively lower predictive performance, with a median below 0.96 and a broader spread of values, indicating variability in accuracy across folds or runs. The intermediate approach, which incorporates decomposition and bagging but omits hyperparameter optimization via EA, exhibits a moderate improvement in both median (approximately 0.97) and stability. Notably, the full EvoBagNet framework achieves the highest median , exceeding 0.99, with minimal variance, highlighting its robustness and superior accuracy. This comparison clearly demonstrates that both decomposition and ensemble learning improve performance over the base learner, but the inclusion of evolutionary hyperparameter tuning further refines the model, significantly boosting accuracy and consistency. For simplicity, the full method name 'EA + Decom + Bag + Cat' is abbreviated as 'Bag-Cat' throughout the text.

5.3 Justification for EvoBagNet's superior performance

The improved prediction ability of the suggested EvoBagNet model compared to conventional machine learning and ensemble approaches traces its origin to its integrated architecture that is well suited to address the complexity associated with financial time series forecasting. What follows is an extensive explanation behind EvoBagNet's experimental superiority.

5.3.1 Multi-scale signal decomposition via CEEMD

Financial time series data, particularly stock prices, are stationary and indicative of multi-frequency processes governed by an array of exogenous and endogenous factors. Conventional forecasting models are likely to struggle with this volatility because they lack the ability to decompose patterns that operate at different temporal scales. EvoBagNet corrects this deficiency by applying CEEMD to isolate the original stock price signal into a family of IMFs. Each IMF extracts a specific frequency component of the data, ranging from short-term fluctuation (high-frequency noise) to long-term market trends. This decomposition enables localized and frequency-aware learning, thereby enhancing the model's ability to capture both micro- and macro-scale dynamics that underlie stock price movements.

5.3.2 Structural diversity ensemble learning

EvoBagNet employs an ensemble architecture with three structurally diverse base learners: Extra Trees, LightGBM, and CatBoost. Their selection is on the basis of their opposing properties for bias-variance trade-offs and learning mechanisms. Extra Trees offers low-bias, high-variance performance with random feature splits. LightGBM offers low-bias, high-variance learning with histogram-based leaf-wise tree growth. CatBoost excels with categorical and noisy features, utilizing ordered boosting. The base model heterogeneity decreases prediction variance and preserves model flexibility, thereby enhancing the ensemble's generalization capability. This multi-view learning approach enables EvoBagNet to learn the heterogeneous nature of stock data more effectively than homogeneous or single-model approaches.

5.3.3 Evolutionary hyper-parameter optimization

Hyperparameter optimization has a direct impact on the performance of ensemble models. Traditional methods, such as grid search or random search, are computationally costly and tend to inefficiently explore high-dimensional

parameter spaces. EvoBagNet has a lightweight, efficient (1 + 1) EA for dynamic hyperparameter adjustment. The algorithm performs mutation-based local optimizations around potential solutions and converges to optimal configurations progressively, following a greedy selection strategy. The application of Gaussian-distributed mutation enables gradient-free search within the parameter space, resulting in efficient convergence and better out-of-sample performance. The adaptive tuning procedure significantly improves the forecasting accuracy and robustness of the ensemble.

5.3.4 Decomposition-aware model fusion

Rather than averaging outputs, EvoBagNet adopts a decomposition-aware fusion strategy. Each decomposed IMF is modelled separately, and the predictions from the ensembles are fused under a validation-error-weighted strategy. This allows the model to assign higher weights to more informative IMFs and suppress the weights of noisy or less informative components. This facilitates a more context-sensitive integration of predictions, making the model stable and accurate under volatile, dynamic market conditions.

5.3.5 Empirical evidence and generalization

EvoBagNet's performance has been established on nine leading IT industry sector shares listed on the National Stock Exchange (NSE), using six performance indicators, including RMSE, MAE, MAPE, and R^2. In each experiment configuration, EvoBagNet performed better than standalone machine learning algorithms (e.g., SVR, Neural Networks, XGBoost) and traditional ensemble methods. Performance scores averaged up to 98.8% accuracy with insignificant standard deviation (± 0.3), indicating the model's high ability to generalize and resilience across various temporal and market scenarios.

6 Limitations and future work

While the EvoBagNet design announces significant advancements in predictive accuracy, robustness, and overallizability compared to traditional methods, certain inherent limitations of the methodology must be acknowledged. These limitations not only establish the limits of the work being worked on now but also provide insight into the lines of future development. CEEMD usage introduces considerable computational overhead. As CEEMD is linked to multiple iterative operations of sifting and ensemble averaging over noisy variants of the signal, time complexity is linearly proportional to the number of IMFs, signal length, and number of realizations. It may be computationally expensive for high-frequency or large datasets. The decomposition itself can be parallelized, though limited resources may pose a problem in real-time or low-latency forecasting applications.

The accuracy and interpretability of CEEMD are highly dependent on parameters such as noise amplitude and the number of ensemble realizations. Improper parameter selection can result in unwanted mode mixing, under- or over-decomposition, or information loss. Despite the use of practical guidelines here to remove uninformative IMFs, adaptive selection of the IMF would also enhance stability and interpretability. EvoBagNet treats each IMF as an independent input stream for base learners. In real life, however, some IMFs may be interrelated or share similar dynamics, particularly when structural breaks or exogenous shocks occur. Such interactions would be ignored, potentially limiting the model's ability to fully leverage inter-IMF relations and yielding suboptimal fusion of the learned representations.

Although EvoBagNet is highly predictive, its hybrid composition–comprising signal decomposition, ensemble learners, and evolutionary optimization–presents challenges to interpretation. In contrast to linear models or shallow trees, EvoBagNet's end-to-end decision process is less transparent and less suited for applications where explainability is a primary concern (e.g., financial regulation or auditability). As with every data-rich approach, the integrity and accuracy of the historical stock data are crucial for EvoBagNet to function effectively. Noisy or

missing records, especially in high-frequency data, can compromise both decomposition and learning from the model. Although imputation and normalization were employed, model performance could still suffer due to data discrepancies or irregular reporting. While the system is capable of supporting exogenous variables (e.g., crude oil prices and exchange rates), this study primarily emphasizes stock-specific historical characteristics. Extended support for macroeconomic signals, news sentiment, or global market indexes could potentially enhance prediction performance but at the expense of increased complexity in data acquisition and preprocessing.

Subsequent studies can overcome these constraints by utilizing more scalable decomposition techniques (e.g., VMD or EWT), adaptive IMF choice based on statistical significance, and explainable ensemble techniques such as SHAP or LIME. Real-time data streams and more generic exogenous characteristics further enhance the applicability of EvoBagNet for high-frequency and dynamic trading conditions.

7 Conclusions

The expanding engagement in stock markets, driven by rapid economic advancements, has intensified the need for accurate stock price prediction to enhance investment outcomes and reduce risks. Traditional statistical and machine learning approaches often fall short due to the unpredictable nature of stock prices, which are characterized by high volatility and dynamic fluctuations. To tackle these challenges, this study presents EvoBagNet, a novel evolutionary bagging ensemble framework designed to deliver robust and reliable stock price predictions.

EvoBagNet leverages a diverse array of nine advanced ML techniques, including tree-based algorithms, neural networks, and ensemble approaches, and applies them to datasets from nine leading IT sector companies. Through extensive experimentation with various train-test splits, EvoBagNet demonstrated outstanding performance, which was evaluated using six different metrics. The model achieved exceptional prediction accuracy, including 97.0% ± 0.7, 98.3% ± 0.5, 97.3% ± 0.8, 97.4% ± 0.6, 97.0% ± 1.0, 98.6% ± 0.4, 98.8% ± 0.4, 91.7% ± 1.2, and 98.4% ± 0.3 for Tech Mahindra, Mindtree, Infosys, Wipro, TCS, Mphasis, L&T Tech, HCL, and Coforge, respectively. These results highlight EvoBagNet's ability to deliver accurate predictions consistently, even for volatile datasets such as HCL.

By addressing the shortcomings of traditional approaches, EvoBagNet stands out as a reliable tool for stock price prediction, offering practical value for investment strategies and risk management. Its demonstrated accuracy and robustness across diverse scenarios establish it as a promising model for real-world applications in dynamic and volatile financial markets. This study underscores the potential of EvoBagNet to drive innovation in financial forecasting and enhance decision-making in complex economic landscapes.

Despite its strong empirical results, EvoBagNet has some limitations. First, the computational cost, while manageable, may increase with larger datasets or deeper ensemble structures. Second, the framework currently operates on historical price and volume-based features without integrating real-time news, sentiment, or macroeconomic indicators, which can influence stock dynamics. Additionally, the use of CEEMD increases preprocessing time and may introduce minor artifacts in some cases if not carefully parameterized.

In future work, several extensions can enhance the scope and practical relevance of this study in the context of stock market prediction. First, incorporating a broader range of input features–such as macroeconomic indicators, financial news sentiment, and global economic variables–may provide a more comprehensive understanding of external factors influencing stock prices, thereby enhancing the model's predictive performance. Second, the application of more advanced deep learning architectures, such as Bidirectional LSTM (Bi-LSTM), Stacked LSTM, and hybrid models like CNN-LSTM, could further capture complex temporal dependencies and nonlinear patterns in financial time series. Third, to assess the robustness and generalizability of the proposed EvoBagNet framework, cross-sectoral evaluations will be conducted using datasets from various industries (e.g., energy, healthcare, finance), ensuring broader applicability across diverse economic domains. Finally, exploring real-time prediction capabilities will be a critical step toward validating the model's effectiveness and computational efficiency under dynamic, live market conditions.

Appendix A: Background of methods

Support vector regression

Support Vector Machine is a well-liked ML technique for classification and regression. SVM Regression also called SVR [64] is a commonly used technique for the prediction and curve fitting for linear as well as non-linear types of regression [65]. It is based on SVM in which support vectors are the points that are closer to the generated hyperplane that segregates the data points about the hyperplane as shown in Fig. 13.

Support Vector Regression aims to identify a function $f(x) = w_i x_i + b$ that approximates the connection between the predictor X and the target feature Y on a given training dataset $D = \{(x_1, y_1),\ (x_2, y_2)...(x_n, y_n)\}$. The ϵ-insensitive tube loss function is introduced by SVR, which states that only those points are considered errors that are beyond the threshold ϵ point. Minimizing the cost function as shown in Eq. (A1) is the main objective of SVR.

$$Minimize: \ \frac{1}{2}||w||^2 \ + \ C\sum_{i=1}^{n}(\xi_i + \ \xi_i^*) \tag{A1}$$

subject to:

$$\forall i: \ y_i - \ w_i x_i \ - b \leq \ \epsilon \ + \ \xi_i$$

$$\forall i: \ w_i x_i + \ b \ - y_i \ \leq \ \epsilon + \ \xi_i^*$$

$$\xi_i \ \xi_i^* \ \geq 0$$

where w and b represents the weight vector and bias term respectively. The regularisation parameter C regulates the trade-off between reducing error and maximizing margin.

Slack variables ξ_i, ξ_i^* deal with hard cases to distinguish exactly. More detailed discussion on SVM and SVR can be found on [66].

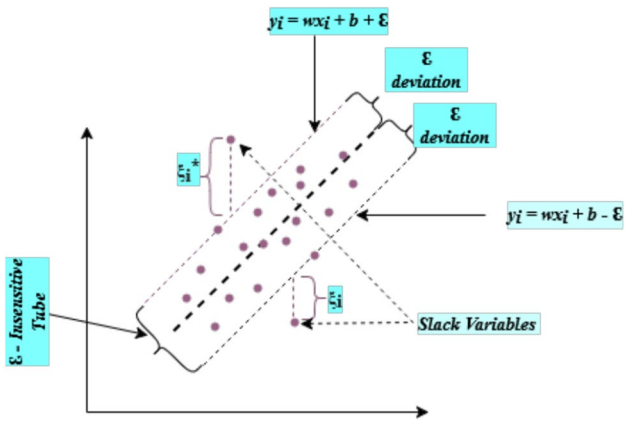


Fig. 13 Support vector regression

Random forest

Several machine-learning applications can make use of decision trees. A tiny bit of noise in the data may cause the tree to grow in an entirely different way [67]. RF tries to overcome this problem using multiple decision trees and training them on different subsets of the feature set at the cost of slightly increased bias, i.e., all the trees in RF see only part of the entire training set [68]. Splitting of training sets into partitions is recursively carried out. The choice of splitting at a particular node depends on some impurity measure like Shannon Entropy or Gini impurity for the classification problem [69]. Stock price prediction is a regression problem, so a measure is needed that shows how much the model's prediction deviates from the actual value.

Some hyper-parameters of random forests need to be optimized for better performance and to control the model from overfitting. These are *n_estimators*, *max_depth*, *max_features*, *min_samples_split* etc.

Lasso regression

The explosive growth of data in modern research and industry has led to an increasing need for efficient methods for feature selection and predictive modelling. Lasso regression is an essential tool in predictive modelling and offers a principled approach to handling high-dimensional data by introducing a penalty term to the ordinary least squares objective function [70]. This approach effectively performs automatic variable selection and reduces overfitting [71]. It has the ability to strike a balance between bias and variance, making it a popular choice across various fields, from biomedical research to finance, where identifying key predictive factors is crucial for effective decision-making. The objective function of Lasso Regression is shown below in Eq. (A2).

$$minimize\left(\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}x_{ij}\beta_j\right)^2+\lambda\sum_{j=1}^{p}|\beta_j|\right) \tag{A2}$$

where y_i represents the observed value for the ith sample, x_{ij} is the value of the jth feature for the ith sample, β_0 is the intercept term, βj is the coefficient for the jth feature, p is the number of features, and λ is the regularization parameter that controls the strength of the penalty term. The key component in lasso regression is the penalty term $\lambda\sum_{j=1}^{p}|\beta_j|$. It penalizes the coefficients' absolute values, causing some of them to decrease towards zero [72]. This favors the solutions in which large number of coefficients are exactly zero, because of this property, lasso regression automatically selects a subset of features and shrinks the others towards zero.

Extreme gradient boosting (XGBoost)

XGBoost, for short, stands for Improvement of Distributed Gradient Boost Decision Tree (GBDT). XGBoost is generally more precise than GBDT and boosts computational speed and performance [73]. In this, features are repeatedly split to add new trees. Adding new trees each time is learning a new function $f_k = (X, \theta_k)$ to fit the residual of the last prediction. When k number of trees are produced after training, each tree has a corresponding leaf node which represents a score. Finally, the recognition prediction value of the sample is calculated by adding the corresponding scores of each tree [74] as shown in Fig. 14.

There are several benefits related to XGBoost that make it ideal for financial research. First and foremost, it is an effective tool for handling massive amounts of data without sacrificing efficiency, which is essential for working with time-series data that is a part of stock market analysis. Secondly, it offers valuable insights into feature significance, facilitating the identification of elements that have the greatest impact on fluctuations in stock prices [75]. The XGBoost objective function is represented as follows:

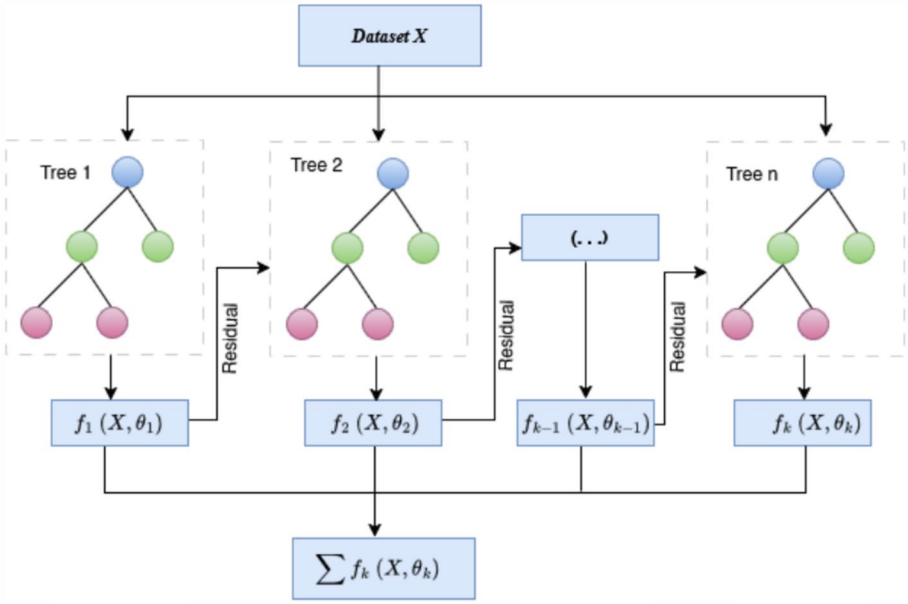


Fig. 14 The training process of XGBoost by the parallel decision trees and residuals

$$f_{\text{obj}} = \sum_s \vartheta\left(y_s, F\left(x_s\right)\right) + \sum_k \Psi\left(f_k\right) \tag{A3}$$

In this context, $\Psi(f)$ refers to the regularization term, T indicates the total number of leaves in the decision tree f; w represents the score associated with leaf j of f; γ, and serves as the threshold for determining whether a split in the decision tree should be performed based on score function improvement.

$$\Psi(f) = \gamma T + \frac{1}{2}\rho \sum_j^T w_j^2 \tag{A4}$$

Additionally, the squared error loss function (SEL) is commonly employed for nonlinear regression tasks. It is mathematically expressed as: $\vartheta(t, y) = (t - y)^2$. XGBoost is an additive ensemble model, trained incrementally by optimizing the objective function. In other words, new decision trees are added one after another, with each tree trying to correct the residual errors of the previous ensemble. At every step, the model updates itself by incorporating the newly built tree in order to enhance the predictive performance and, at the same time, regularizes in order to avoid overfitting. Mathematically, this can be described as:

$$f_{\text{obj}}^k = \sum_s \vartheta\left(y_s, F_{k-1}\left(x_s\right) + f_k\left(x_s\right)\right) + \sum_k \Psi\left(f_k\right) \tag{A5}$$

The final XGBoost model employed for estimating concrete strength is formulated as follows, where N represents the total number of decision trees in the ensemble.

$$F_{XGB} = \sum_{k=1}^{N} f_k(x) \tag{A6}$$

Author contributions Conceptualization, U.B., K.S., V.M., and A.M.U.D.K. A.G.; methodology, U.B., K.S., V.M., A.M.U.D.K., and M.N.; software, U.B., K.S., V.M., and A.M.U.D.K.; validation, U.B., V.M., and A.M.U.D.K.; formal analysis, U.B., V.M., and A.M.U.D.K.; investigation, U.B., K.S., V.M., A.M.U.D.K., and M.N.; resources, U.B., V.M., and A.M.U.D.K.; data curation, U.B., K.S., V.M., and A.M.U.D.K.; writing—original draft preparation, U.B., V.M., A.M.U.D.K., and M.N.; writing—review and editing, U.B., K.S., V.M., A.M.U.D.K., and M.N.; visualization, U.B., A.M.U.D.K., and M.N.; supervision, K.S., V.M., and A.M.U.D.K.; project administration, K.S., V.M., and A.M.U.D.K.; funding acquisition, U.B., K.S., and V.M.. All authors have read and agreed to the published version of the manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions. This work was supported by the ASPIRE Award for Research Excellence (AARE-2020). (ASPIRE award number AARE20-100 grant #21T055)

Data availability Data will be available on request to akibkhanday@uaeu.ac.ae; umar.bashir@jammuuniversity.ac.in and code can be found at https://github.com/Akibkhanday/; https://github.com/UmarBashir131/.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Hasan F, Al-Okaily M, Choudhury T, Kayani U (2024) A comparative analysis between fintech and traditional stock markets: using Russia and Ukraine war data. Electron Commer Res 24(1):629–654
2. Jiang W (2021) Applications of deep learning in stock market prediction: recent progress. Expert Syst Appl 184:115537
3. Yuan X, Yuan J, Jiang T, Ain QU (2020) Integrated long-term stock selection models based on feature selection and machine learning algorithms for China stock market. IEEE Access 8:22672–22685
4. Yañez C, Kristjanpoller W, Minutolo MC (2024) Stock market index prediction using transformer neural network models and frequency decomposition. Neural Comput Appl 36:1–21
5. Idrees SM, Alam MA, Agarwal P (2019) A prediction approach for stock market volatility based on time series data. IEEE Access 7:17287–17298
6. Xia H, Weng J, Boubaker S, Zhang Z, Jasimuddin SM (2024) Cross-influence of information and risk effects on the IPO market: exploring risk disclosure with a machine learning approach. Ann Oper Res 334(1):761–797
7. Al-Khasawneh MA, Raza A, Khan SUR, Khan Z (2024) Stock market trend prediction using deep learning approach. Comput Econ 45:1–32. https://doi.org/10.1007/s10614-024-10714-1
8. Sheth D, Shah M (2023) Predicting stock market using machine learning: best and accurate way to know future stock prices. Int J Syst Assur Eng Manag 14(1):1–18
9. Kong X, Chen Z, Liu W, Ning K, Zhang L, Muhammad Marier S, Liu Y, Chen Y, Xia F (2025) Deep learning for time series forecasting: a survey. Int J Mach Learn Cybern 16:1–34
10. Ge Q (2025) Enhancing stock market forecasting: a hybrid model for accurate prediction of s&p 500 and CSI 300 future prices. Expert Syst Appl 260:125380
11. Yang J, Li P, Cui Y, Han X, Zhou M (2025) Multi-sensor temporal fusion transformer for stock performance prediction: an adaptive sharpe ratio approach. Sensors 25(3):976
12. Vishwakarma VK, Bhosale NP (2024) A survey of recent machine learning techniques for stock prediction methodologies. Neural Comput Appl 37:1–22
13. Lin C-T, Wang Y-K, Huang P-L, Shi Y, Chang Y-C (2022) Spatial-temporal attention-based convolutional network with text and numerical information for stock price prediction. Neural Comput Appl 34(17):14387–14395
14. Billah MM, Sultana A, Bhuiyan F, Kaosar MG (2024) Stock price prediction: comparison of different moving average techniques using deep learning model. Neural Comput Appl 36(11):5861–5871
15. Shaban WM, Ashraf E, Slama AE (2024) Smp-dl: a novel stock market prediction approach based on deep learning for effective trend forecasting. Neural Comput Appl 36(4):1849–1873
16. Bhuriya D, Kaushal G, Sharma A, Singh U (2017) Stock market predication using a linear regression. In: 2017 international conference of electronics, communication and aerospace technology (ICECA), vol 2. IEEE, pp 510–513
17. Dash RK, Nguyen TN, Cengiz K, Sharma A (2023) Fine-tuned support vector regression model for stock predictions. Neural Comput Appl 35(32):23295–23309
18. Bansal M, Goyal A, Choudhary A (2022) Stock market prediction with high accuracy using machine learning techniques. Proc Comput Sci 215:247–265
19. Singh G (2022) Machine learning models in stock market prediction. arXiv preprint arXiv:2202.09359

20. Jain S, Kain M (2018) Prediction for stock marketing using machine learning. Int J Recent Innov Trends Comput Commun 6:131–135
21. Hemanth D et al (2021) Stock market prediction using machine learning techniques. Adv Parallel Comput Tech Appl 40:331
22. Zhang D, Lou S (2021) The application research of neural network and bp algorithm in stock price pattern classification and prediction. Futur Gener Comput Syst 115:872–879
23. Vijh M, Chandola D, Tikkiwal VA, Kumar A (2020) Stock closing price prediction using machine learning techniques. Proc Comput Sci 167:599–606
24. Mustaffa Z, Sulaiman MH (2023) Stock price predictive analysis: An application of hybrid barnacles mating optimizer with artificial neural network. Int J Cogn Comput Eng 4:109–117
25. Zhong X, Enke D (2019) Predicting the daily return direction of the stock market using hybrid machine learning algorithms. Financ Innov 5(1):1–20
26. Hao Y, Gao Q (2020) Predicting the trend of stock market index using the hybrid neural network based on multiple time scale feature learning. Appl Sci 10(11):3961
27. Khan W, Ghazanfar MA, Azam MA, Karami A, Alyoubi KH, Alfakeeh AS (2022) Stock market prediction using machine learning classifiers and social media, news. J Ambient Intell Human Comput 13:1–24
28. Yu Y, Dai D, Yang Q, Zeng Q, Lin Y, Chen Y (2025) An intelligent framework based on optimized variational mode decomposition and temporal convolutional network: applications to stock index multi-step forecasting. Expert Syst Appl 268:126222
29. Li L, Shan K, Geng W (2025) Forecasting crude oil price using secondary decomposition-reconstruction-ensemble model based on variational mode decomposition. J Futures Mark 45:1601
30. Li J, Chen W, Zhou Z, Yang J, Zeng D (2024) Deepar-attention probabilistic prediction for stock price series. Neural Comput Appl 36:1–18
31. Li S, Xu S (2024) Enhancing stock price prediction using GANs and transformer-based attention mechanisms. Empir Econ 68:1–31
32. Xu Y, Zhang Y, Liu P, Zhang Q, Zuo Y (2024) Gan-enhanced nonlinear fusion model for stock price prediction. Int J Comput Intell Syst 17(1):12
33. Xie L, Wan R, Wang Y, Li F (2024) Stock closing price prediction based on ICEEMDAN-FA-BiLSTM-GM combined model. Int J Mach Learn Cybern 16:1–25
34. Lu W, Li J, Wang J, Qin L (2021) A CNN-BiLSTM-AM method for stock price prediction. Neural Comput Appl 33(10):4741–4753
35. Chen W, Zhang H, Mehlawat MK, Jia L (2021) Mean-variance portfolio optimization using machine learning-based stock price prediction. Appl Soft Comput 100:106943
36. Sharma D, Sarangi PK, Sahoo AK et al (2023) Analyzing the effectiveness of machine learning models in nifty50 next day prediction: a comparative analysis. In: 2023 3rd international conference on advance computing and innovative technologies in engineering (ICACITE). IEEE, pp 245–250
37. Ballings M, Poel D, Hespeels N, Gryp R (2015) Evaluating multiple classifiers for stock price direction prediction. Expert Syst Appl 42(20):7046–7056
38. Basak S, Kar S, Saha S, Khaidem L, Dey SR (2019) Predicting the direction of stock market prices using tree-based classifiers. North Am J Econ Financ 47:552–567
39. Yetis Y, Kaplan H, Jamshidi M (2014) Stock market prediction by using artificial neural network. In: 2014 world automation congress (WAC). IEEE, pp 718–722
40. Shen J, Shafiq MO (2020) Short-term stock market price trend prediction using a comprehensive deep learning system. J Big Data 7:1–33
41. Senol D, Ozturan M (2009) Stock price direction prediction using artificial neural network approach: the case of turkey. J Artif Intel 1:70
42. Nabipour M, Nayyeri P, Jabani H, Shahab S, Mosavi A (2020) Predicting stock market trends using machine learning and deep learning algorithms via continuous and binary data; a comparative analysis. IEEE Access 8:150199–150212
43. Kamalov F (2020) Forecasting significant stock price changes using neural networks. Neural Comput Appl 32(23):17655–17667
44. Bathla G, Rani R, Aggarwal H (2023) Stocks of year 2020: prediction of high variations in stock prices using lstm. Multimed Tools Appl 82(7):9727–9743
45. Sivadasan E, Mohana Sundaram N, Santhosh R (2024) Stock market forecasting using deep learning with long short-term memory and gated recurrent unit. Soft Comput 28(4):3267–3282
46. Elminaam A, Salama D, El-Tanany AMM, El Fattah MA, Salam MA (2024) Stockbilstm: utilizing an efficient deep learning approach for forecasting stock market time series data. Int J Adv Comput Sci Appl 15(4):446
47. Bühlmann P (2012) Bagging, boosting and ensemble methods. Handbook of computational statistics: concepts and methods. Springer, Cham, pp 985–1022

48. Khan AA, Chaudhari O, Chandra R (2024) A review of ensemble learning and data augmentation models for class imbalanced problems: combination, implementation and evaluation. Expert Syst Appl 244:122778
49. Breiman L (1996) Bagging predictors. Mach Learn 24:123–140
50. Bühlmann P, Yu B (2003) Boosting with the l 2 loss: regression and classification. J Am Stat Assoc 98(462):324–339
51. Neshat M (2020) The application of nature-inspired metaheuristic methods for optimising renewable energy problems and the design of water distribution networks. PhD thesis
52. Neumann F, Wegener I (2007) Randomized local search, evolutionary algorithms, and the minimum spanning tree problem. Theoret Comput Sci 378(1):32–40
53. Torres ME, Colominas MA, Schlotthauer G, Flandrin P (2011) A complete ensemble empirical mode decomposition with adaptive noise. In: 2011 IEEE international conference on acoustics, speech and signal processing (ICASSP). IEEE, pp 4144–4147
54. Zhang Y, Yan B, Aasma M (2020) A novel deep learning framework: prediction and analysis of financial time series using CEEMD and lstm. Expert Syst Appl 159:113609
55. Chen Y, Zhao P, Zhang Z, Bai J, Guo Y (2022) A stock price forecasting model integrating complementary ensemble empirical mode decomposition and independent component analysis. Int J Comput Intell Syst 15(1):75
56. Liu Y, Liu X, Zhang Y, Li S (2022) CEGH: a hybrid model using CEEMD, entropy, GRU, and history attention for intraday stock market forecasting. Entropy 25(1):71
57. Flores BE (1986) A pragmatic view of accuracy measurement in forecasting. OMEGA Int J Manag Sci 14:93–98
58. Kvålseth TO (1985) Cautionary note about R^2. Am Stat 39(4):279–285
59. Maçaira PM, Cyrino Oliveira FL (2016) Another look at SSA. Boot forecast accuracy. Int J Energy Stat 4(02):1650008
60. Cui C, Wang P, Li Y, Zhang Y (2023) McVCsb: a new hybrid deep learning network for stock index prediction. Expert Syst Appl 232:120902
61. Kara Y, Boyacioglu MA, Baykan ÖK (2011) Predicting direction of stock price index movement using artificial neural networks and support vector machines: the sample of the Istanbul stock exchange. Expert Syst Appl 38(5):5311–5319
62. Di Persio L, Honchar O et al (2016) Artificial neural networks architectures for stock price prediction: comparisons and applications. Int J Circuit Syst Signal Process 10:403–413
63. Kulkarni S (2023) Impact of various data splitting ratios on the performance of machine learning models in the classification of lung cancer. In: Proceedings of the 2nd international conference on emerging trends in engineering (ICETE 2023), vol 223. Springer, pp 96
64. Jin S (2024) A comparative analysis of traditional and machine learning methods in forecasting the stock markets of China and the US. Int J Adv Comput Sci Appl 15(4):1–8
65. Parbat D, Chakraborty M (2020) A python based support vector regression model for prediction of COVID19 cases in India. Chaos Solitons Fractals 138:109942
66. Chhajer P, Shah M, Kshirsagar A (2022) The applications of artificial neural networks, support vector machines, and long-short term memory for stock market prediction. Decision Anal J 2:100015
67. Elsayed N, Abd Elaleem S, Marie M (2024) Improving prediction accuracy using random forest algorithm. Int J Adv Comput Sci Appl 15(4):436
68. Yin L, Li B, Li P, Zhang R (2023) Research on stock trend prediction method based on optimized random forest. CAAI Trans Intell Technol 8(1):274–284
69. Polamuri SR, Srinivas K, Mohan AK (2019) Stock market prices prediction using random forest and extra tree regression. Int J Recent Technol Eng 8(1):1224–1228
70. Li J, Chen W (2014) Forecasting macroeconomic time series: lasso-based approaches and their forecast combinations with dynamic factor models. Int J Forecast 30(4):996–1015
71. Rastogi A, Qais A, Saxena A, Sinha D (2021) Stock market prediction with lasso regression using technical analysis and time lag. In: 2021 6th international conference for convergence in technology (I2CT). IEEE, pp 1–5
72. Lee JH, Shi Z, Gao Z (2022) On lasso for predictive regression. J Econom 229(2):322–349
73. Sharma P, Jain MK (2023) Stock market trends analysis using extreme gradient boosting (xgboost). In: 2023 international conference on computing, communication, and intelligent systems (ICCCIS). IEEE, pp 317–322
74. Chen T, Guestrin C (2016) XGBoost: a scalable tree boosting system. In: Proceedings of the 22nd Acm Sigkdd international conference on knowledge discovery and data mining, pp 785–794
75. Saetia K, Yokrattanasak J (2022) Stock movement prediction using machine learning based on technical indicators and google trend searches in Thailand. Int J Financ Stud 11(1):5

Authors and Affiliations

Umar Bashir[1] · Kuljeet Singh[4] · Vibhakar Mansotra[1] · Akib Mohi Ud Din Khanday[3] · Mehdi Neshat[2]

✉ Mehdi Neshat
mehdi.neshat@uts.edu.au

Umar Bashir
umar.bashir@jammuuniversity.ac.in

Kuljeet Singh
kuljeet.singh@jammuuniversity.ac.in

Vibhakar Mansotra
vibhakarmansotra@jammuuniversity.in

Akib Mohi Ud Din Khanday
akibkhanday@gmail.com

[1] Department of Computer Science and IT, University of Jammu, Jammu Tawi, Jammu 180006, India

[2] Faculty of Engineering and Information Technology, University of Technology Sydney, Ultimo, Sydney, NSW 2007, Australia

[3] Department of Information Technology, Cluster University of Srinagar, Gogji Bagh, Srinagar, J&K, India

[4] Department of Computer Science and Engineering, Chitkara University, Punjab, India